

Highlights

LIR+LVM: Bounded Execution of LLM-Generated Code on Microcontrollers

Yingjie Ma, Mengjia Ma, Sheng Zhao, Peng Liu

- Structured IR with small grammar for stable LLM-to-MCU code generation.
- ~2 KB VM with capability gating and step-limited bounded execution.
- Withstands 8,000 adversarial payloads with zero unauthorized hardware access.
- 9.8× smaller payload than JSON; 48.7% lower interpretation energy.
- Same bytecode runs on ESP8266 and STM32F103 with <50-line adapter.

LIR+LVM: Bounded Execution of LLM-Generated Code on Microcontrollers[★]

Yingjie Ma^{a,1}, Mengjia Ma^{a,2}, Sheng Zhao^{a,3} and Peng Liu^{a,*,4}

^aNorth China Institute of Aerospace Engineering (NCIAE), No. 133 Aimin East Road, Langfang, 065000, Hebei, China

ARTICLE INFO

Keywords:

LLM-based edge intelligence
Security
Energy efficiency
Hardware control
Bytecode virtual machine
Bounded execution
IoT

ABSTRACT

Large language models can generate control programs for microcontrollers, the most resource-constrained endpoint of LLM-based edge intelligence, but the output must compress to tens of bytes, execute in kilobytes of RAM, and avoid unauthorized hardware access or unbounded execution. Existing approaches address only one side: format validators ensure syntactic correctness but provide no runtime safety, whereas sandboxed runtimes such as WebAssembly exceed typical MCU memory budgets. We present LIR + LVM, a two-layer framework that decouples generation from execution: the LLM emits LIR, a structured intermediate representation, that a deterministic compiler lowers to bytecode; the LVM, a lightweight virtual machine on the MCU, enforces capability gating and step limits before executing. We implement on ESP8266 and STM32F103 and evaluate along three axes. For security, the system blocks unauthorized access across 8,000 adversarial payloads. For efficiency, the bytecode is 9.8× smaller than optimized JSON and halves interpretation energy. For reliability, the pipeline achieves full pass rate on deterministic tasks, and sampling achieves 0.982 reliability on stochastic tasks. The same bytecode runs on both architectures with under 50 lines of adapter code.

[★]Code: <https://github.com/MYJ-GOD/lir-lvm>

^{*}Corresponding author

 1730316713@163.com (Y. Ma)

 <https://github.com/MYJ-GOD/lir-lvm> (Y. Ma)

ORCID(s):

1
2
3
4

1. Introduction

Large language models (LLMs) have demonstrated strong code generation capabilities [12, 25, 34]. A natural next step is to push this capability to the edge: having LLMs generate control programs for microcontrollers (MCUs) that directly drive sensors and actuators [26, 7]. This vision introduces a fundamental tension that existing systems do not face. Desktop-class targets such as robot arms [26] and embodied agents [7] run Python or action tokens on hardware with megabytes of RAM and an operating system to catch faults. An MCU, by contrast, has no OS-level sandbox, a few kilobytes of SRAM, and I/O pins wired directly to relays and sensors. A control program must therefore satisfy three constraints simultaneously: it must be small enough to traverse a bandwidth-constrained link (tens of bytes, not hundreds), it must execute within the device's memory budget, and, because it originates from an untrusted LLM, it must not be allowed to run unboundedly or drive hardware that the operator never authorized. On a bare-metal MCU driving physical actuators, a hallucinated instruction that writes to an unauthorized relay is not merely a software bug; it is a safety incident. This tension defines LLM-based edge intelligence: the edge device must execute untrusted LLM output with no OS sandbox and kilobytes of memory.

Existing approaches each address only one side of this three-way tension (Fig. 1). Grammar-constrained decoding [18] ensures that the LLM's output is syntactically well-formed, but provides no runtime safety once the program reaches the device. High-level languages such as Python are easy for LLMs to generate but require a full interpreter (30–300 KB RAM) with no formal execution boundaries. Compact binary formats such as WebAssembly [9] provide sandboxed execution but need 30–60 KB RAM (exceeding the budget of most MCUs) and are difficult for LLMs to generate directly. JSON interfaces, the most common LLM output format, produce hundreds of bytes per instruction. No existing approach provides a representation that LLMs can generate reliably, compresses to tens of bytes for transmission, and bounds execution on the device.

We propose a two-layer framework that addresses all three requirements, guided by six design goals defined in Section 3.2. The first layer is LIR (a structured intermediate representation) that uses a small, C-like grammar oriented toward LLM output. The second layer is compact, varint-encoded bytecode optimized for transmission and on-device execution. A deterministic compiler bridges the two: the LLM generates LIR text, and the compiler lowers it to bytecode with no ambiguity and no information loss.

To make this concrete, consider a task that would be natural in a smart-farm deployment: “turn on the water pump and read the water level sensor.” The LLM emits six lines of IR (declaring the required devices, writing a relay, waiting 500 ms, reading a sensor, and halting). The compiler lowers these to 14 bytes of bytecode. On the MCU, the virtual machine checks that `relay1` and `water_sensor` were explicitly declared before any I/O is allowed, then executes the seven instructions in sequence. If the LLM had

hallucinated a write to an undeclared device (e.g., `relay2`), the capability gate would block it before the pin is touched. If the program contained an infinite loop, the step limit would terminate it after a fixed number of operations. It illustrates the three properties the framework must provide: compact representation (14 bytes for a task that JSON encodes in over a hundred), bounded execution (every I/O gated, every program terminates), and generation stability (a grammar small enough that the LLM rarely deviates). Section 3.2 develops it in full. These design choices also reduce on-device interpretation energy (Section 7.2.2), which is particularly relevant for battery-constrained edge devices where control-program updates are frequent.

Three technical challenges arise. First, the *generation–execution gap*: LLMs generate structured text well, but compact bytecode is unreadable to models, and the IR-to-bytecode lowering must be deterministic. Second, *on-device bounded execution*: AI-generated programs are untrusted and must be constrained via capability gating, step limits, and structural validation. Third, *closed-loop feedback and repair*: execution failures must produce structured errors that the LLM can use for self-correction. This third challenge is what distinguishes a static code generator from an adaptive edge intelligence agent: the readback–compare–retry mechanism enables the LLM to observe the device's actual state and repair its program accordingly, forming a closed-loop control cycle rather than a fire-and-forget deployment.

Our contributions are as follows:

- **A bounded-execution VM with capability gating and step limits** (Section 4). The LVM (lightweight virtual machine) is a compact stack machine with a restricted 14-instruction set; its small semantic surface makes determinism, progress, and bounded termination provable by case analysis (Propositions 1–3). Because each privileged I/O dispatches through the capability gate, unauthorized access is blocked by a membership test before each I/O rather than by heuristic detection.
- **A two-layer control representation with a deterministic compiler** (Section 3): LIR + bytecode decouples generation friendliness from execution friendliness, with the compiler's termination, determinism, and totality established in Proposition 1 and its consistency with the VM confirmed on 204 verified outputs in Proposition 2.
- **Cross-platform pipeline evaluation** (Section 7). The pipeline achieves full pass rate on deterministic tasks and high reliability on random tasks. Best-of- K ($K=3$) sampling restores reliability to 0.982 under stochastic generation. The same bytecode and its safety guarantees execute on both ESP8266 (Xtensa) and STM32F103 (ARM Cortex-M3) with fewer than 50 lines of I/O-adaptor code changed.

The remainder of this paper is organized as follows. Section 2 frames the design space. Section 3 presents the

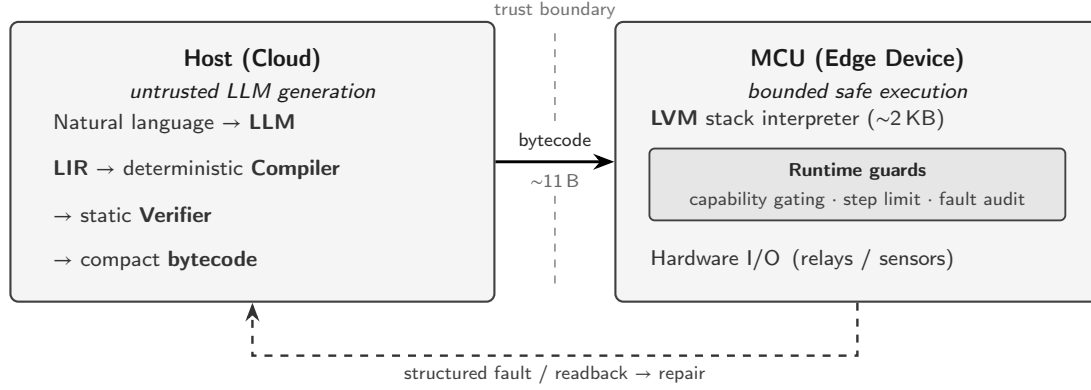


Figure 1: LIR + LVM system architecture. The two-layer design decouples generation (NL → LLM → LIR → bytecode) from execution (LVM → hardware I/O), connected by a deterministic compiler with runtime protection mechanisms.

efficiency-driven design: LIR and its compilation to compact bytecode. Section 4 covers the security-driven design: the LVM threat model, bounded executor, capability gating, and static verifier. Section 5 formalizes the operational semantics and proves the safety and compiler–VM consistency properties. Section 6 analyzes each attack surface against those guarantees, and Section 7 presents the experimental evaluation. Section 8 discusses design trade-offs and scope, and Section 9 reviews related work. Section 10 concludes.

2. Background and Design Space

Driving hardware from LLM output involves three layers: (i) *generation*, where the LLM produces a structured program from natural language; (ii) *representation*, where that program is encoded for transmission; and (iii) *execution*, where the encoded program runs on the target device. On resource-constrained MCUs, each layer faces distinct constraints that jointly shape the design space, and no existing representation satisfies all three at once. This section frames those constraints, then surveys how prior work occupies the design space along the generation, representation, and execution axes.

2.1. Problem Space and Design Axes

The three layers translate into three design axes. *Generation friendliness* depends on the surface syntax the LLM must emit: formats abundant in pre-training data (e.g., C-like assignment statements) yield lower error rates than formats rare in pre-training (e.g., raw hexadecimal bytecode or WASM S-expressions). *Payload density* is determined by the representation that travels to the MCU over a potentially bandwidth-constrained link (e.g., 200 bps on NB-IoT, ~300 bps on LoRa). Self-describing formats such as JSON carry structural overhead (braces, quotes, and key names) that inflates the payload beyond its semantic content. *Execution safety* depends on what the device can enforce. A microcontroller such as the ESP8266 (Xtensa LX106, 80 KB RAM) runs firmware in a bare-metal loop with I/O through memory-mapped registers. Because the program originates from an untrusted LLM, the runtime must gate I/O to declared

Table 1

Design-space positioning of representative approaches and embedded VMs.

Approach	Gen.	Payload (B)	RAM (KB)	Cap.	Step-lim.	CL
<i>Representative approaches</i>						
JSON + zlib	✓	106.7	N/A [†]	×	×	×
CBOR	×	105.2	N/A [†]	×	×	×
Arduino C	✓	188.1	~30	×	×	×
Grammar-constrained	✓	—	—	×	×	×
<i>Embedded VMs</i>						
TinyVM [21]	×	—	~1	×	×	×
Darjeeling [8]	×	—	~4	×	×	×
MicroPython	✓	~80–150	30–300	×	×	×
AtomVM [2]	×	—	~16	×	×	×
WAMR (full) [9]	×	~50–100	30–60	×	gas [‡]	×
WAMR (minimal) [§]	×	—	~15	×	gas [‡]	×
WAMR + Outlines	✓ [¶]	~50–100	30–60	×	gas [‡]	×
LIR + LVM (this work)	✓	10.9	~2	✓	✓	✓

[†]Requires a host-side interpreter; RAM cost is application-dependent. [‡]WAMR provides gas-metering (functionally equivalent to step-limiting) but no application-level capability gating; an authenticated WASM module can access any memory-mapped I/O within its linear memory. [§]WAMR minimal configuration on ARM Cortex-M (interpreter mode, no JIT); the 15 KB figure excludes application heap and stack. [¶]With caveats: WAT uses S-expressions with explicit type annotations, more error-prone for LLMs than LIR's C-like syntax; no published data on LLM generation success rate for WAT. ^{||}Core VM RAM only; framework overhead depends on platform (see Table 4).

devices, bound execution against infinite loops, and report structured faults for closed-loop repair.

These axes are in tension. Text formats such as JSON generate reliably but are too verbose for the link. Grammar-constrained decoding secures the generation axis but says nothing about execution safety. Dense binary formats execute efficiently but neither generate reliably nor stay within the device memory budget. These tensions exist in any code deployment scenario, but they become critical when LLMs generate control policies for edge systems: the programs must traverse bandwidth-constrained links, execute on MCUs with no OS-level sandboxing, and avoid activation errors with physical safety consequences. Table 1 positions representative approaches along these axes. Each occupies one or two corners. No existing approach is generation-friendly, compact on the wire, and bounded on the device. LIR and its compiled bytecode are designed for exactly this combination.

2.2. Structured LLM Generation

Grammar-constrained decoding [18] and tools such as Outlines [42] ensure syntactic validity during LLM inference, but they stop at the token boundary: they provide no downstream compilation, no capability declarations, and no bounded execution. A parallel line drives hardware from generated code directly (Python policies for robot arms [26], fine-tuned vision-language models for action output [7, 22]), but these target desktop-class robots rather than MCUs. Recent work extends LLM code generation toward embedded and IoT systems [15, 28], yet none integrate a generation-targeted IR with formal bounded execution on the device. LIR + LVM targets this gap.

2.3. Embedded VMs and Their Gaps

Lightweight embedded VMs (TinyVM [21], Darjeeling [8], AtomVM [2], and WASM/WAMR [9]) provide bytecode interpreters with an emphasis on generality, portability, or sandbox isolation, but none treats LLM generation cost as a design objective. Unlike WASM/WAMR, LIR runs within ~2 KB RAM while natively expressing capability declarations and readback/retry. Embedded scripting runtimes (MicroPython [19], eLua [29]) need 6–300 KB and lack capability gating or step-limiting. Composing existing tools, WASM/WAMR for sandboxed execution and grammar-constrained decoding for format validity, leaves three gaps: (1) WAT is not an LLM-friendly target (S-expressions with explicit type annotations); (2) WAMR's sandbox offers no application-level capability gating; (3) the combination provides no integrated closed-loop feedback. Table 1 positions these approaches, showing that the LVM is the only runtime providing both capability gating and step-limit guarantees within a 2 KB RAM footprint.

Across the three axes above, prior work covers one or two concerns; LIR + LVM is designed to address all three.

3. Efficiency-Driven Design: LIR and Compilation

3.1. Two-Layer Representation

Following Section 1, we adopt a two-layer representation. The first layer is LIR, a structured control representation oriented toward LLM output; the second is a variant-encoded bytecode stream that the compiler generates and the LVM executes. LIR caters to stable generation and ease of debugging; bytecode is optimized for small transmission size and low-overhead execution. A deterministic compiler bridges the two. The overall system architecture is shown in Fig. 1: a frontend LLM generation pipeline (NL → LIR → compiler → verifier → bytecode), a backend MCU execution path (transmission → LVM → hardware I/O → feedback → repair), and a runtime protection layer (capability gating, step-limit, fault audit, closed-loop verification) enforced during execution.

3.2. LIR Design Goals

The system must satisfy six design goals, spanning the representation, the compiler, and the runtime:

- *Representation*: token efficiency (minimal generation cost) and payload density (small transmission size);
- *Compiler/verifier*: deterministic compilation (reproducible lowering) and static verifiability (pre-execution checking);
- *Runtime*: bounded execution (guaranteed termination) and fault auditability (structured diagnostics).

At the LIR level, the representation goals translate to four concrete requirements: LIR must be:

1. Smaller than JSON;
2. Easier to generate and repair than raw hex bytecode;
3. Deterministically lowerable to bytecode;
4. Capable of explicitly expressing capability, wait, readback, and retry semantics.

Design choices: LIR adopts C-like assignment statements rather than S-expressions or stack-based representations. This choice is motivated by the abundance of C-like syntax in LLM pre-training data, which benefits stable generation. The one-statement-per-line format facilitates line-by-line normalization and human debugging. `wait` accepts a millisecond argument to maintain consistency with hardware timing semantics. `retry` uses a fixed count instead of conditional predicates, trading expressiveness for implementation simplicity. The current statement subset covers deterministic control, temporal sequencing, closed-loop maintenance, and error recovery. Conditional branching is indirectly supported through `retry`'s implicit readback mechanism.

Running example. Consider the task “turn on the water pump and read the water level sensor.” The LLM emits six lines of LIR: first declaring the required devices (`relay1`, `water_sensor`), then issuing a relay write, a 500 ms delay, a sensor read, and a halt. Each statement maps deterministically to one or two bytecode instructions: capability declarations become `GTWAY`, writes become `LIT+IOW`, reads become `IOR`, and waits become `WAIT`. This pattern (declare capabilities, actuate, wait, sense, halt) is the canonical structure of Tasks 1–7 in our deterministic evaluation set (Section 7.1). The full grammar that defines this mapping is given by the following EBNF:

Listing 1: LIR EBNF grammar

```

<program> ::= "task" <identifier> "{" <stmt>+ "}"
<stmt>    ::= <cap_decl> | <set_stmt> | <read_stmt>
           | <wait_stmt> | <halt_stmt>
           | <readback_stmt> | <retry_stmt>
<cap_decl> ::= "require" "cap" "(" <device> ")"
<set_stmt>  ::= "set" <device> "=" <bit>
<read_stmt> ::= "read" <device>
<wait_stmt> ::= "wait" <number> "ms"
<halt_stmt> ::= "halt"
<readback_stmt> ::= "readback" <device> "expect" <bit>
<retry_stmt> ::= "retry" <number> "times" "{" <stmt>+ "}"
<device>    ::= "water_sensor" | "temperature_sensor"
               | "humidity_sensor" | "relay1" | "relay2"
<bit>       ::= "0" | "1"
<number>    ::= [1-9][0-9]*

```


The current grammar subset supports seven statement types (see EBNF grammar). The EBNF grammar enumerates device names explicitly for clarity; a production deployment would use a parameterized `<identifier>` rule with a separately maintained device registry, enabling new device types without grammar changes. The LLM-generated LIR for the running example is:

Listing 2: Running example: LLM-generated LIR

```
task water_pump {
  require cap(relay1)
  require cap(water_sensor)
  set relay1 = 1
  wait 500ms
  read water_sensor
  halt
}
```

Device name to device_id mapping: water_sensor (1), temperature_sensor (2), humidity_sensor (3), relay1 (5), relay2 (6).

3.3. Compact Bytecode Design Principles

Compact bytecode achieves its small footprint and bounded execution through four design principles, each targeting a distinct constraint of the MCU deployment scenario:

1. varint / zigzag encoding for minimal payload size;
2. a stack-based execution model for deterministic, allocation-free interpretation;
3. capability-gated I/O for privilege confinement;
4. a unified fault / pc / steps return format for structured diagnostics.

3.4. Instruction Set and Bounded-Execution Profile

The LVM is a general-purpose stack machine. Its host-side reference implementation provides roughly fifty instructions spanning control flow, arithmetic, arrays, and debugging (Appendix C.4), but most are unnecessary for deterministic MCU control: categories such as recursion, unbounded loops, dynamic allocation, and division introduce termination or memory behavior that a single step counter cannot bound. For untrusted on-device actuation we therefore deploy a 14-instruction *bounded-execution profile* (Table 5): constants and stack manipulation (LIT, DUP, DRP, SUB), equality and control flow for closed-loop retry (EQ, JZ, JMP, READBACK, RETRY), capability-gated I/O (GTWAY, IOW, IOR, WAIT), and termination (HALT). This profile is the subset whose safety properties are formalized in Section 5, deployed to ESP8266 and STM32F103 firmware, and exercised by every experiment in this paper. The remaining instructions are implemented and unit-tested in the host VM but lie outside the on-device safety analysis, for the reasons made precise in Section 5.4.

Table 2

Fair four-scenario payload comparison (113 tasks, avg. bytes).

Scenario	LIR side	JSON side	Advantage
Raw LLM output	108.0 B	285.9 B	2.6×
+zlib (level 9)	86.1 B	150.8 B	1.8×
Cloud-compiled vs. optimized JSON [†]	10.9 B	106.7 B	9.8×
Cloud-compiled vs. raw-LLM JSON [‡]	10.9 B	150.8 B	13.8×

[†] LIR compiled on cloud; JSON uses hand-crafted minimal schema (156.4 B raw → 106.7 B zlib, Appendix Table 15). [‡] LIR compiled on cloud; JSON sent as zlib-compressed raw LLM output (285.9 B → 150.8 B).

3.5. Encoding Advantages

A fair compression comparison distinguishes *what the LLM generates* (LIR or JSON text) from *what the MCU executes* (compiled bytecode). Table 2 presents four scenarios covering both transmission and execution perspectives.

Table 2 shows that LIR text is inherently smaller than JSON text (2.6× raw, 1.8× after zlib) due to its minimal keyword-based syntax. Under matched optimization, bytecode versus a hand-optimized minimal-schema JSON (156.4 B raw → 106.7 B zlib, Appendix Table 15), the ratio is 9.8×, which is the most directly comparable baseline since both sides are optimized. Against zlib-compressed *raw-LLM* JSON (285.9 B → 150.8 B), the advantage grows to 13.8×. Even in the most conservative scenario (raw text without compilation or compression), LIR remains 2.6× smaller because its grammar removes JSON structural overhead (braces, quotes, key names). Against the binary IoT standard CBOR (105.2 B, Appendix Table 15), the bytecode achieves a 9.6× compression ratio.

The practical implication is deployment time. On an NB-IoT link at 200 bps, transmitting 10.9 B of bytecode takes approximately 0.44 s, versus 4.27 s for 106.7 B of zlib-JSON, a 3.8 s difference per program. For a field device that receives new control programs periodically (e.g., updated irrigation schedules), this difference determines whether an update completes within a single radio wake window or requires a multi-second connection that drains the battery budget. The 9.8× ratio is a direct proxy for deployment feasibility on energy-constrained links.

Additional encoding baselines (CBOR: 105.2 B, MessagePack: 97.42 B, compact JSON: 127.18 B) are reported in Appendix C.2. All subsequent analyses use the cloud-compiled scenario (bytecode vs. zlib-JSON) as the primary reference, since the two-layer architecture is designed for cloud-to-MCU deployment.

Fig. 2 illustrates the LIR-to-bytecode lowering for the running example, and Table 3 lists the main lowering relationships for the experimental subset.

For readback and retry, the minimal experimental chain is fully implemented and performs real lowering: readback compiles to IOR + LIT + EQ, and retry compiles to explicit control flow composed of LIT/DUP/JZ/JMP/SUB/DRP. The compiler also retains a structured execution-plan output for debugging and subsequent board-level closed-loop runner reuse.

In the formal model READBACK and IOR share identical stack semantics (both push dev(d) given d ∈ cap); in the implementation readback is lowered to IOR ; LIT ; EQ instead of

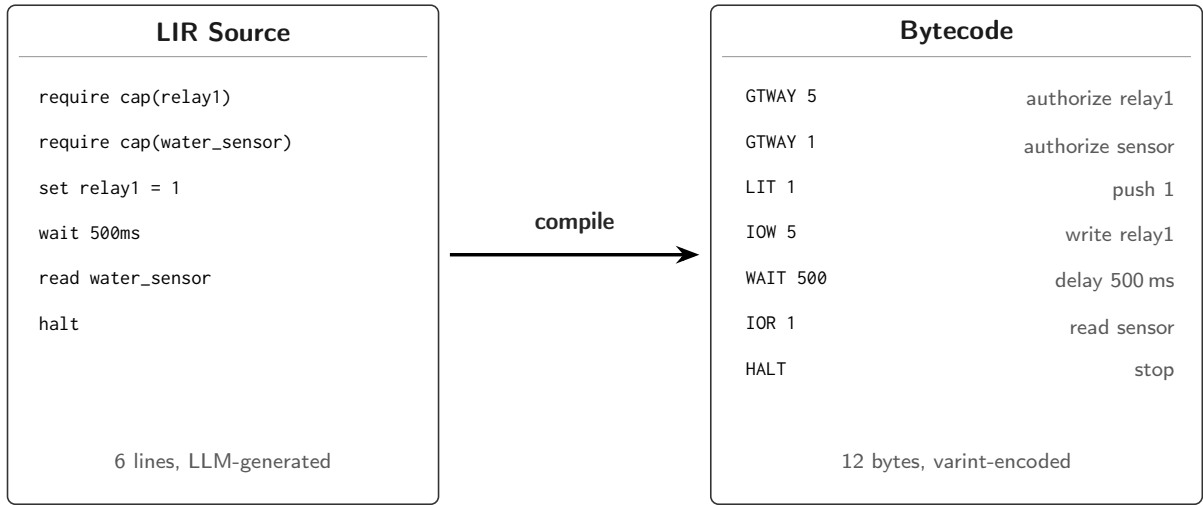


Figure 2: LIR-to-bytecode lowering for the running example.

Table 3

Main LIR-to-bytecode lowering rules.

LIR statement	Lowering result
require cap(relay1)	GTWAY 5
require cap(water_sensor)	GTWAY 1
set relay1 = 1	LIT 1 ; IOW 5
set relay1 = 0	LIT 0 ; IOW 5
set relay2 = 1	LIT 1 ; IOW 6
read water_sensor	IOR 1
read temperature_sensor	IOR 2
read humidity_sensor	IOR 3
read relay1	IOR 5
wait 500ms	WAIT 500
halt	HALT

carrying a dedicated opcode, while the structured execution plan retains the “closed-loop verification read” intent. This lets fault reports separate a “sensor read” from a verification read at the plan level, so the repair loop can identify which stage of the set→readback→compare→retry chain failed.

3.6. Compiler Properties

Our compiler C lowers LIR text to bytecode. Let LIR^* be the set of all valid LIR programs, and BC^* be the set of all valid bytecode sequences. By design, LIR’s EBNF grammar is effectively LL(1) at the token level: each statement starts with a unique keyword, with no left recursion or ambiguity. The only lexer-level ambiguity, the prefix overlap between read and readback, is resolved by the lexer before handing tokens to the parser, leaving the parser itself strictly LL(1). This is a property of the grammar as designed and is realized by the recursive-descent parser in the released implementation. Given it, C ’s basic properties follow from standard parser-generator theory (we state them as a design-level property rather than a machine-checked theorem):

Property 1 (Termination, determinism, and totality).

For every input p , C terminates in $O(n)$ time where n is the token count. Moreover, C is deterministic ($|C(p)| = 1$ for all valid p) and total, mapping every input (valid or

invalid) to either a bytecode sequence or a structured error (LIR_PARSE_ERROR).

Expression power boundary. The current LIR v0.1 subset covers seven statement types (see EBNF grammar and Table 3): capability declaration, device set/read, timed wait, halt, readback verification, and retry loops. Constructs outside v0.1 (arbitrary if/else branching, nested loops, general arithmetic, and user-defined functions) and the scalability implications of extending the grammar are discussed in Section 8.1.

The compiler establishes only that valid LIR lowers to valid bytecode; it says nothing about bytecode that bypasses the compiler. The compactness secured on the wire must therefore be paired with a static verifier and runtime guards that confine *any* bytecode stream, compiler-produced or adversarial, before it can drive hardware.

4. Security-Driven Design: The LVM Bounded Executor

The LVM is the backend that executes untrusted bytecode on the device. Its design is driven by an explicit threat model, which we state first. The remaining subsections then introduce the mechanisms that answer it: a bounded instruction profile, capability gating, step-limiting, and a static verifier.

4.1. Threat Model

We take *runtime bounded execution* as the security scope: an LLM-generated program must not escalate privileges, run unboundedly, or produce an un-audited fault before it can drive hardware. Link encryption, supply-chain integrity, physical tampering, and side-channel attacks (timing, power) are outside the scope of this work, consistent with prior embedded-VM solutions. These are composed from orthogonal mechanisms. We assume the compiler and static verifier are trusted (i.e., they faithfully implement the EBNF grammar and jump-bounds check), and that the MCU hardware and I/O drivers function correctly.

The trusted computing base comprises the static verifier, the LVM executor, the I/O drivers, and the MCU hardware. Everything upstream of the verifier is untrusted: in particular, we treat the LLM and its output as fully adversarial, since a model may hallucinate, be poisoned, or be steered by prompt injection. Defense relies on three complementary layers: (1) the EBNF parser intercepts format deviations (around 70% of observed LLM errors); (2) the static verifier checks opcode validity, varint integrity, and jump-target bounds before any execution; and (3) the LVM runtime enforces capability gating and step-limits during execution. We consider four threat actors against this stack: *malicious LLM output* (an unauthorized or malformed program), *encoding-layer attacks* (bytecode crafted to bypass the parser entirely), *prompt injection* through the feedback channel, and *side-channel inference* from observable fault behavior. The attack surfaces these actors expose are analyzed against the formal guarantees in Section 6.

4.2. LVM Architecture

The LVM interprets and executes the bytecode on the MCU. The primary implementation targets ESP8266 (Xtensa LX106). Cross-platform validation on STM32F103 (ARM Cortex-M3) confirms platform-independent execution (Section 7.5). The LVM state comprises:

```
pc / stack / locals / ret-stack /
call-depth / steps / fault
```

and provides a unified response:

1. OK: [result varint][steps varint][0x01]
2. FAULT: [fault varint][pc varint][0x00]

Execution is a single loop over a stack machine. Each iteration first checks the step-limit ($steps \geq L$) and the stack bound, then fetches the opcode at pc, increments the step counter, dispatches on the opcode, and advances pc (or redirects it for JMP/JZ). The loop has no recursion and no heap allocation: the operand stack is a fixed-size array, locals are a fixed slot vector, and every operation either pushes/pops a bounded number of slots or performs one capability-checked I/O. This structure lets the same ~430-line core run unmodified on both an 80 KB-RAM Xtensa device and a 10 KB-RAM Cortex-M3, with only the GPIO/sensor adapter changed. Every exit is either the OK response (top-of-stack result and consumed step count) or the FAULT response (structured fault code and faulting pc), so the host can always distinguish “ran to completion” from “stopped by a guard” and feed the latter into the repair loop (Section 7.1.1).

The quantified resource footprint of the LVM is summarized in Table 4.

4.3. Static Verifier

The host-side static verifier is the first line of defense: it rejects malformed bytecode before transmission, so a faulty stream never consumes device bandwidth or execution cycles. Verification proceeds in two passes over the decoded

Table 4

LVM resource footprint (measured on ESP8266 and STM32F103).

Metric	ESP8266	STM32F103C6	Measurement method
Flash (LVM core / with framework [†])	~4 KB / —	~4 KB / ~21 KB	Arduino IDE compilation
RAM (stack + vars)	~2 KB	~2.7 KB	Build report + runtime watermarking
Max stack depth	16 levels	64 levels	Deepest nested test
Code lines (C++)	~430	~430	Shared core, platform I/O adapters
WAMR min config	30–60 KB	—	WAMR documentation [9]

[†]STM32F103C6 has 32 KB flash. The 21 KB figure includes LVM core logic (~4 KB) plus STM32duino framework overhead and DHT sensor library; the LVM core logic itself is identical across platforms.

RAM is measured via the build report (.data+.bss) and runtime stack watermarking.

instruction stream. The first pass reconstructs instructions from the varint encoding and rejects any byte sequence that does not decode to a known opcode carrying a well-formed operand; an unsupported opcode, a truncated varint, or a device identifier outside the valid range each aborts decoding with a bad-encoding fault. The second pass examines the control-flow skeleton: every jump offset is resolved to an absolute target, and a target that falls outside the program bounds is rejected. Resolving jumps statically guarantees that the program counter cannot leave the loaded program once execution begins, which removes an entire class of out-of-bounds behavior from the runtime. Because both passes complete on the host before the payload is sent, the device receives only bytecode whose opcodes decode and whose jumps land in range, leaving the runtime guards to handle the faults that depend on execution state rather than on structure.

4.4. Runtime Protection Mechanisms

The runtime guards handle the faults that static verification cannot anticipate, because they depend on values the program computes as it runs. Four guards operate during the execution loop, each confining a distinct fault class. Capability gating is the central mechanism: the LVM admits a device into an authorization set only when an explicit gateway instruction names it, and every subsequent write or read checks membership in that set before touching hardware. An I/O to a device that was never authorized raises a capability fault instead of driving the pin, so the capability gate blocks unauthorized access via a membership test before each I/O rather than by heuristically detecting suspicious patterns after the fact. The step-limit guard bounds total work: the counter advances by one per instruction and, on reaching the limit L , forces termination, so a backward jump or a retry loop either converges within a finite number of steps or stops safely at the bound. The stack guard rejects both an overflow that would exceed the fixed operand-stack capacity and an underflow that would pop an empty stack, the latter being the typical symptom of a truncated or adversarial program. A program-counter bound complements the verifier’s static jump check by catching any residual out-of-range fetch at runtime. When a guard fires, it returns a structured fault carrying the fault class and the faulting program counter rather than failing silently. This keeps the runtime safe to expose to untrusted

Table 5

Compact bytecode instruction set (14-instruction formalized subset).

Instr.	Opcode	Operand	Stack effect	Steps
LIT	0x1e	value (svarint)	push value	1
GTWAY	0x50	1B dev_id	—	1
IOW	0x46	1B dev_id	pop, write	1
IOR	0x47	1B dev_id	push read	1
WAIT	0x51	ms (uvarint)	—	1
HALT	0x52	—	—	1
EQ	0x2c	—	pop a,b; push (a==b)	1
JZ	0x65	offset (svarint)	pop, jmp if 0	1
JMP	0x64	offset (svarint)	—	1
DUP	0x40	—	push top	1
SUB	0x33	—	pop a,b; push (b-a)	1
DRP	0x41	—	pop, discard	1
READBACK [†]	(lowered)	1B dev_id	push dev state	1
RETRY [†]	(lowered)	count, offset	dec counter; jmp if >0	1

[†]READBACK and RETRY are logical instructions of the formal model (Section 5); the compiler lowers them to sequences over the twelve primitive opcodes above (readback $\rightarrow$ IOR ; LIT ; EQ ; retry $\rightarrow$ a LIT/DUP/JZ/JMP/SUB/DRP loop), so they carry no dedicated bytecode opcode. Operand widths are variable-length: unsigned args use uvarint, while LIT values and jump offsets use zigzag-encoded svarint.

output, and gives the repair loop (Section 7.1.1) a precise signal to act on.

Capability gating also provides a form of structural privacy. **Structural privacy of generated programs.** It bounds not only what a program can activate but also what it can leak. A concern specific to LLM-based edge intelligence is that LLMs can memorize and reproduce sensitive training data such as credentials or secrets [10]. In our setting this surface is structurally narrow: LIR programs cannot express free-form strings or secrets, since the grammar (Listing 1) admits only a fixed device vocabulary, bit values, and integer durations, and the compiler rejects any token outside this alphabet. A leaked credential therefore cannot survive compilation to bytecode, and capability gating further bounds the *effect* of any generated program to the declared device set. What LIR does *not* address is leakage in the natural-language prompt or in the LLM’s hidden state, nor confidentiality of the bytecode in transit (link encryption is out of scope); these are orthogonal protections that compose with LIR rather than being subsumed by it.

The complete instruction set for the experimental subset is listed in Table 5, with each instruction’s opcode, operands, stack effect, and step count.

Each instruction consumes exactly one step: the LVM increments the step counter by 1 for every instruction executed, triggering a fault when $k \geq L$. For program instruction count n , actual execution steps are $\leq \min(n, L)$; loops composed of JZ/JMP or RETRY still increment k per iteration, so they either terminate within finite steps or raise a step-limit fault. With the runtime guards in place, we now analyze how they respond to each attack surface.

5. Formal Safety Guarantees

This section establishes three safety properties of the LVM execution model (determinism, progress, and bounded

termination) and the consistency between a compiled program and its execution. Because the LVM restricts itself to 14 fixed-step instructions (no unbounded loops, no dynamic allocation, no instruction whose cost depends on its operands), each property reduces to case analysis over a finite rule set and a monotone step counter. The formalization follows a small-step operational style; the complete machine configuration, 17 transition rules, and full proofs are given in Appendix C.1–B.

5.1. Execution Model

We model LVM execution as a state transition system. A configuration $\sigma = \langle p, pc, s, dev, cap, k, L, f, r \rangle$ comprises the bytecode program p , program counter pc , operand stack s , device state dev , capability set cap , step counter k , step limit L , fault status f , and retry counter r . Every instruction advances k by 1; when $k \geq L$, the STEPGUARD rule forces a STEP fault. I/O instructions (IOW, IOR, READBACK) check $d \in cap$ before dispatching; if the check fails, a CAP fault is raised. Invalid opcodes trigger an OPC fault. The full 17-rule transition system is closed over the 14-instruction set of Table 5.

5.2. Safety Properties

We establish three properties that together guarantee the LVM cannot harm the device it controls. In plain terms: the VM always terminates (it cannot run forever), every I/O is checked against a declared allow-list (it cannot touch unauthorized hardware), and the same program always produces the same result (execution is predictable). These properties hold because the instruction set is deliberately restricted: every instruction costs exactly one step, there is no unbounded loop instruction, and every I/O goes through a capability check. The formal statements follow.

Proposition 1 (Determinism). *For any configuration σ , if $\sigma \xrightarrow{1} \sigma_1$ and $\sigma \xrightarrow{1} \sigma_2$, then $\sigma_1 = \sigma_2$.*

Proposition 2 (Progress). *For every non-terminal configuration σ with $f = \perp$ and $k < L$, there exists σ' such that $\sigma \xrightarrow{1} \sigma'$.*

Proposition 3 (Bounded execution). *Starting from $\sigma_0 = \langle p, 0, \epsilon, dev_0, cap_0, 0, L, \perp, r_0 \rangle$: (1) σ_0 reaches a terminal configuration in at most L steps; (2) any IOW d or IOR d with $d \notin cap$ triggers a CAP fault; (3) any invalid opcode triggers an OPC fault; (4) the retry counter r is strictly decreasing per loop iteration, and total execution steps remain bounded by L .*

Intuitively: (1) holds because every rule increments k by exactly 1, so STEPGUARD forces termination within L steps; (2)–(3) hold because the first unauthorized I/O or invalid opcode immediately raises its fault; (4) holds because each retry iteration decrements r while k keeps rising. Complete proofs are in Appendix B.

Proposition 3(2)–(3) has a direct practical consequence for the security metric used in the evaluation. We define the Unauthorized Access Blocking Rate (UABR) as the fraction

of unauthorized I/O requests that are blocked before reaching hardware. Within the formal model, this rate is exactly 1: every unauthorized request is caught by the capability check, with no dependence on how the program was generated.

Corollary 1 (UABR lower bound, within the model).

Under the threat model (Section 4.1), every unauthorized request raises a CAP fault, so $UABR(r) = 1$ at the model level. This follows directly from Proposition 3(2)–(3). The empirical UABR in Section 7.3.1 measures the implementation’s agreement with this model-level bound.

This bound is a direct consequence of checking $d \in cap$ before each IOW/IOR/READBACK, not an emergent property: it holds independent of whether the generating LLM is honest, hallucinating, or adversarial, and carries over to the synthetically adversarial payloads of Section 7.3.2, which never pass through the LLM at all.

5.3. Compiler–VM Consistency

The safety properties above bound what the LVM does with *any* bytecode. Consistency closes the loop back to the compiler of Section 3, establishing that a compiled program executes with the semantics its LIR source intended.

Property 2 (Compiler–VM consistency). *For each grammar production in the LIR EBNF, the lowering rule of Table 3 preserves the intended semantics. By case analysis over the seven statement types (full analysis in Appendix B), each lowering is a direct structural encoding of its EBNF production: e.g., set $d = v$ lowers to `LIT v ; IOW d`, where rule `LIT` pushes v and rule `IOW-OK` writes it to $dev(d)$; `retry n` times lowers to a loop with decreasing counter r , where rules `RETRY-LOOP/RETRY-EXHAUST` guarantee exactly n iterations. Full-program equivalence $\llbracket p \rrbracket_{LIR} = \llbracket C(p) \rrbracket_{LVM}$ follows by induction on the statement sequence, using the stack-balance invariant: every statement-level lowering leaves the operand stack at the same height it found it, so sequential composition preserves both device state dev and capability set cap across statement boundaries. This is a hand proof over a fixed seven-production grammar, corroborated empirically: across 204 verified compiler outputs (113 deterministic + 91 verified closed-loop outputs, Appendix Table 13) the LVM device state matches the expected effect in every case.*

5.4. Scope

The formal model covers the 14 instructions of the bounded-execution profile (Table 5), which is exactly the subset deployed to firmware and exercised by the evaluation. The broader instruction categories of the host VM (Appendix C.4), namely functions and recursion, general loops, dynamic memory, and division, are excluded because each introduces behavior that a single step counter cannot bound. The LIR v0.1 grammar restricts generation to the same subset, so the representation, firmware, and formal model coincide on one bounded surface. The three properties follow from two design choices, a finite fixed-step instruction set and a capability check before each I/O, and the complete proofs are in Appendix B.

5.5. Closed-Loop and Host VM

The current system supports the closed-loop chain set $\rightarrow readback \rightarrow compare \rightarrow retry$, giving the LLM a unified observation-and-repair interface. At runtime the LVM follows a fixed five-stage path (loading, decoding, static checking, runtime guarding, and output) in which any guard violation transitions immediately to a terminal Fault state. Fig. 3 illustrates the closed-loop verification state machine.

Remark 1 (Running example revisited). For the running example introduced in Section 1 and compiled in Section 3.2 (“turn on the water pump and read the water level sensor”), the compiled 7-instruction sequence executes in 7 steps. With the default step limit $L = 1000$, this consumes only 0.7% of the budget, illustrating that the theoretical bound (Proposition 3) provides a $> 140\times$ safety margin over typical benign programs. This margin is by design: the step limit protects against adversarial programs, not typical ones.

6. Security Analysis

We now reason through each attack surface exposed by the threat model, showing how the runtime guards and formal guarantees mitigate them. The empirical validation (per-guard ablation and an 8,000-payload fuzzing campaign) follows in Section 7.3.

6.1. Attack Surfaces and Defenses

Unauthorized activation. An adversary may try to drive a relay or pump the operator never authorized, the most consequential attack on physical hardware. Each IOW/IOR/READBACK dispatches through the capability gate: if the target device is absent from the declared set cap , the rule raises a CAP fault and the configuration becomes terminal before a write reaches the driver. Proposition 3(2) makes this independent of the input source: an unauthorized access raises a CAP fault whether or not the program was generated honestly. This is the basis of the $UABR = 1$ lower bound (Corollary 1).

Invalid and injected opcodes. A malformed or hostile stream may contain bytes that decode to no valid instruction. The static verifier rejects such streams before execution. Any opcode that nonetheless reaches the interpreter triggers the OPCF rule and an OPC fault (Proposition 3(3)). The two layers are deliberately redundant: structurally malformed bytecode is caught statically, while a decodable-but-invalid opcode is caught at runtime.

Non-terminating execution. An adversary may craft backward jumps or retry loops intended to cause non-termination or exhaust device resources. Because every rule, including `JMP`, `JZ`, and `RETRY`, increments the step counter k by exactly one, `STEPGUARD` forces a `STEP` fault after at most L steps (Proposition 3(1)). The bound is on steps executed rather than on the program’s syntactic shape, so it holds regardless of how the control flow is arranged. On a microcontroller wired to physical actuators, this converts a potential stuck-actuator incident into a clean, audited halt.

Malformed encoding. An adversary may craft a bytecode stream with truncated variants, out-of-bounds jump targets,

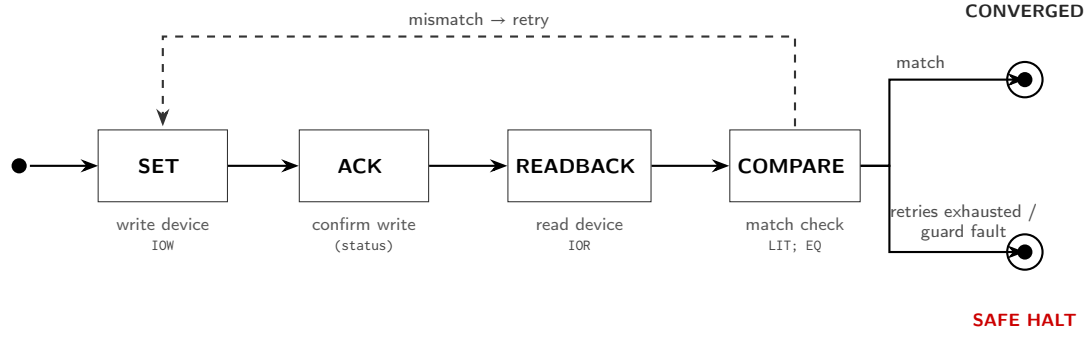


Figure 3: State-machine model of closed-loop verification.

or stack-violating sequences. The static verifier intercepts structurally malformed bytecode before transmission. Any decodable-but-invalid stream that reaches the runtime triggers the POPF rule or stack-bound checks. Because the two layers cover structurally malformed and decodable-but-invalid streams respectively, no adversarial stream can bypass both.

Adversarial instructions and prompt injection. LLM-based edge intelligence exposes two injection surfaces: the natural-language prompt itself and the closed-loop feedback channel. An attacker may embed malicious directives in the user instruction (e.g., instructing the LLM to write to an unauthorized device or enter an infinite loop), or inject adversarial sensor values or fault text through the feedback path to steer subsequent generation. LIR + LVM mitigates the *consequence* of both vectors through defense-in-depth. The EBNF grammar constrains the LLM’s output to a fixed alphabet of statements (set/read/wait/halt/retry on declared device names), so a semantically malicious instruction cannot survive compilation if it requires constructs outside the grammar. The static verifier rejects malformed bytecode before transmission. Capability gating ensures that even a syntactically valid but adversarial program cannot access undeclared devices. The step limit bounds total execution regardless of control-flow arrangement. The key property is that each layer operates on a different artifact (source text, bytecode structure, runtime state), so bypassing one layer does not bypass the others. A prompt-injection attack that succeeds at the LLM level, convincing the model to emit a malicious but syntactically valid program, still faces the same capability and step-limit guarantees as any other untrusted program (Corollary 1). What the architecture does *not* defend against is a prompt injection that causes the LLM to emit a syntactically valid, capability-compliant program with *incorrect semantics* (e.g., writing 1 to a relay when the user intended 0); this class of attack requires semantic validation beyond the VM’s scope.

Side-channel inference. The LVM returns a unified fault format (fault code, program counter, status byte) regardless of which guard fired, so an observer cannot distinguish the fault class from the response shape alone. Timing and power side channels are out of scope.

Physical-safety boundary. UABR measures blocking of unauthorized VM-level access requests; it does not imply

physical safety. A single *authorized* IOW instruction can still activate a pump or relay, so physical safety requires external hardware interlocks (watchdog timers, current limiters) beyond the VM’s scope. The guarantee is that the set of *possible* actuations is bounded to the declared capability set, not that every authorized actuation is physically benign.

The qualitative arguments above are validated empirically in the next section, where a per-guard ablation and an 8,000-payload fuzzing campaign stress the combined boundary across the attack classes enumerated here.

7. Evaluation

Our evaluation answers four questions, aligned with the security and efficiency axes that motivate this work:

- **Q1 (Efficiency):** How small is the LIR/bytecode representation relative to JSON, CBOR, and Arduino C, across transmission payload, generation tokens, and on-device execution energy (Section 7.2)?
- **Q2 (Security):** Does the LVM contain the output of an untrusted LLM (or an adversary), and does each guard contribute independently (Section 7.3)?
- **Q3 (Reliability):** How reliably does the pipeline generate correct programs across temperatures, models, and task complexities, and on tasks derived from real-world IoT deployments (Section 7.4)?
- **Q4 (Portability):** Does the same bytecode and its safety guarantees execute unchanged across MCU architectures (Section 7.5)?

We address them in turn, with the experimental setup common to all of them described next.

7.1. Experimental Setup

LLM. All experiments used LLaMA 3.1 8B [14] as the primary model, served via Ollama on a local server. Multi-model comparison (Appendix C.2, Table 14) additionally evaluated Qwen3 8B and DeepSeek-R1 8B [20] as local models, and DeepSeek-V4-Pro via commercial API to validate cross-architecture generalization. Temperature settings varied by experiment: temperature = 0.0 for deterministic baselines, 0.0–1.0 for temperature scanning. Apart from temperature, all sampling parameters were left at the Ollama defaults (top-p 0.9, top-k 40, repetition penalty 1.1); only the temperature was set explicitly, and no fixed random seed was pinned for LLM generation. Consequently, temperature > 0 runs are not

bit-for-bit reproducible; we therefore report the mean and standard deviation over three independent passes for every stochastic setting (Table 9).

Hardware. The primary MCU target was ESP8266 (80 KB RAM, Xtensa LX106) running Arduino framework. The LVM firmware (~430 lines C++, ~2 KB RAM) was compiled via Arduino IDE. Cross-platform validation was performed on STM32F103C6T6 (ARM Cortex-M3, 32 KB flash, 10 KB RAM) using the STM32duino board package, confirming that LVM’s platform-independent C++ core requires only I/O adapter changes (<50 lines) for porting. End-to-end LLM generation experiments were conducted on ESP8266. STM32 validation confirmed correct execution of compiled bytecode (capability-gated I/O, sensor reads, relay control) via serial communication.

Task sets. We used five task sets (all included in the repository; see the data availability statement): (1) 113 deterministic tasks covering 7 control patterns; (2) 96 closed-loop tasks covering 12 categories; (3) 15 control-flow tasks (if/else, repeat); (4) 600 randomly generated tasks across 12 categories (50 per category); (5) 29 tasks derived from real-world IoT deployments (Home Assistant, Tasmota, ESPHome, Arduino/ESP projects) covering 20 control pattern categories.

Metrics. We evaluated using the following metrics, defined formally to ensure reproducibility:

Compression ratio:

$$R_{\text{AVG}} = \frac{1}{|T|} \sum_{t \in T} \frac{B_{\text{JSON}}(t)}{B_{\text{compact}}(t)},$$

$$R_{\text{WEIGHTED}} = \frac{\sum_t B_{\text{JSON}}(t)}{\sum_t B_{\text{compact}}(t)}.$$

Task Success Rate (TSR):

$$\text{TSR} = \frac{1}{|T|} \sum_{t \in T} \mathbf{1}[t \text{ passes all stages}],$$

with 95% Wilson confidence intervals.

Unauthorized Access Blocking Rate (UABR): $\text{UABR} = N_{\text{blocked}}/N_{\text{total}}$, where N_{blocked} is the number of unauthorized requests blocked by capability gating or the validator.

Safe Convergence Rate (SCR): the fraction of tasks reaching the target state after k retry rounds.

First-success@k: the fraction of tasks whose first successful generation occurs at round k (non-cumulative; distinguishes from pass@k which is cumulative).

Experiment overview. We organized experiments along three axes:

- **(A) Efficiency:** compression ratio and RTT of bytecode vs. JSON.
- **(B) Security:** effectiveness of capability gating, step-limit, validator, and fault audit (UABR, fault distribution, SAFE shunting).
- **(C) Reliability:** whether the LLM→LIR→bytecode pipeline produces correct programs, evaluated on 113 deterministic tasks covering seven control patterns (single

write, single read, write-then-wait-then-halt, wait-then-read, write-wait-read, multiple reads, and pulse).

Core results (temperature=0.0). On the 113-task set, each pipeline stage (IR validity, compilation, verification, execution, and task success) achieved 1.000 (95% Wilson CI: [0.967, 1.000]). Of the 113 tasks, 100 required no repair, and the remaining 13 were corrected in the second round, with no failures.

Additional experiments. We evaluate three further dimensions (detailed results in Appendix C.2): (i) *Closed-loop control:* retry raises SCR under both paths, and the LIR path achieves lower RTT than JSON under the same retry strategy. (ii) *Token cost:* LIR reduces output tokens by 51% versus JSON; under prompt caching, its incremental cost is roughly half of JSON’s (Table 12). (iii) *Random task coverage:* on 600 randomly generated tasks, the success rate is 0.970, with 7 of 12 categories achieving 100% (Section 7.4.4). (iv) *Arduino C baseline:* LLM-generated Arduino C produces 17.3× larger payloads than the bytecode.

Fig. 4 consolidates the headline success rates and their 95% Wilson confidence intervals across all task sets, so the relative precision of each estimate (driven by sample size) can be read at a glance: the large deterministic and random sets yield tight intervals, while the smaller real-world and closed-loop subsets carry wider ones.

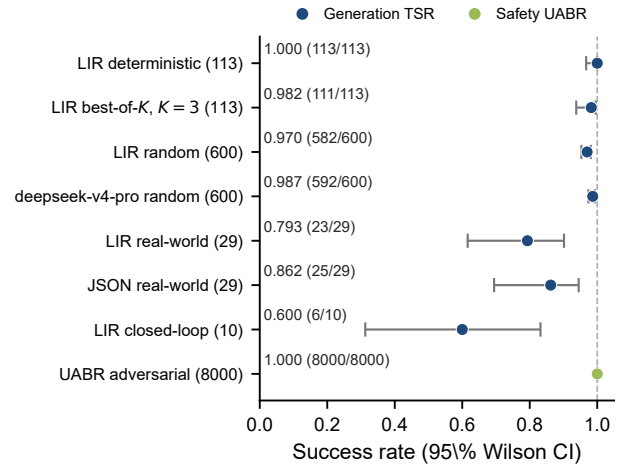


Figure 4: Headline success rates with 95% Wilson confidence intervals across all task sets (forest plot).

7.1.1. Pipeline and Repair Strategy

The complete generation chain is:

NL → LIR → Compiler → Verifier → LVM/Sim → Feedback → Repair

Taking the running example from Section 1 (“turn on the water pump and read the water level sensor”): the LLM outputs 6 lines of LIR, the compiler lowers it to 14 bytes of bytecode (GTWAY 5; GTWAY 1; LIT 1; IOW 5; WAIT 500; IOR 1; HALT), the verifier passes static checks, and the LVM completes execution on ESP8266 in 7 steps. The pipeline

never silently discards a failed attempt: whichever stage rejects the program returns a structured diagnostic, and that diagnostic is what closes the loop back to the LLM.

The feedback is deliberately structured rather than free-form. Each failure is reported as the stage that rejected the program, the typed error code raised there, and a short corrective hint keyed to that code. The hint is not a generic “try again”: an unknown-device error tells the model to draw only from the declared device list, a missing-capability error tells it to authorize the device before its first access, and an invalid-argument error tells it to keep actuator values binary and waits non-negative. Because the typed codes partition the failure space and each maps to one targeted instruction, the model receives a precise correction rather than an opaque rejection, which lets a single repair round resolve most errors. Sensor readings observed during closed-loop execution are fed back through the same channel, so the model repairs against the device’s actual state rather than its assumed one.

Two properties make the repair loop safe to run on untrusted output. First, it terminates: repair is capped at three rounds, after which the accumulated fault codes are returned without execution, and within that budget the information available to the model grows monotonically as each round appends its diagnostic to the prior context, so the loop cannot oscillate indefinitely. Second, repair never weakens the safety boundary: every regenerated program re-enters the same static verifier and the same runtime guards as a first attempt, so a repaired program enjoys exactly the capability-gating and step-limit guarantees of Section 4.4. The repair loop improves the success rate without ever becoming a path around the guards.

Repair cost. Repair incurs negligible overhead: among the 113 deterministic tasks, the 13 that failed on the first attempt were all corrected in a single repair round (95% Wilson CI [0.772, 1.000]).⁵ The typed feedback (an undeclared device, a missing capability declaration, an out-of-range argument) maps each failure to one corrective hint, so the model rarely needs more than one targeted nudge to converge.

Model robustness. To assess whether LIR generation reliability depends on a specific model, we evaluated four models on the 113-task deterministic set at temperature = 0.0 (full results in Appendix Table 14). **llama3.1:8b** and **deepseek-r1:8b** both achieve TSR = 1.000; **deepseek-v4-pro** (commercial API) achieves 0.991 (one parse error from C-style semicolons). **qwen3:8b** achieves 0.903, indicating that LIR generation reliability depends on instruction-following capability and is not universal across 8B models. These results show that LIR’s advantage lies at the *representation* level (payload compactness, bounded execution with formal guarantees), not the *model* level: stronger models reduce parse errors but do not eliminate the payload gap or execution safety gap that motivates LIR. The 0.097 TSR gap for qwen3:8b is

consistent with a grammar-constrained decoding fix, not an LIR redesign.

7.2. Efficiency: Payload, Energy, and Token Cost

This subsection answers Q1 by measuring the representation along three efficiency dimensions (transmission payload, on-device execution energy, and generation token cost) against JSON, CBOR, and Arduino C.

7.2.1. Compression and Transmission Latency

LIR text is consistently smaller than JSON text across all 113 tasks (Fig. 5), with a weighted ratio of 2.6× and a per-task median of 2.5×. The advantage comes from LIR’s minimal keyword-based syntax, which eliminates the braces, quotes, and key names that JSON carries. Compiling LIR to bytecode before transmission raises the ratio to 9.8× over optimized JSON (Table 2), because varint-encoded opcodes replace self-describing textual keys with positional integers. Against Arduino C, the advantage reaches 17.3× (Section 7.2.4), reflecting the payload cost of general-purpose syntax when only control logic is needed.

The payload-size advantage translates directly into transmission latency, where low-bandwidth links amplify it. The following figures are estimates based on raw payload size; protocol handshaking, fragmentation, and ARQ overhead are not modeled. On NB-IoT (200 bps, e.g., ESP32 + SIM7000), transmitting 10.9 B of bytecode takes approximately 0.44 s, versus 4.27 s for 106.7 B of zlib-JSON (3.83 s difference); on LoRa (~300 bps, e.g., ESP32-C3 + SX1276), the difference is approximately 2.55 s. Payload reduction thus translates directly into shorter transmission windows on bandwidth-constrained links.

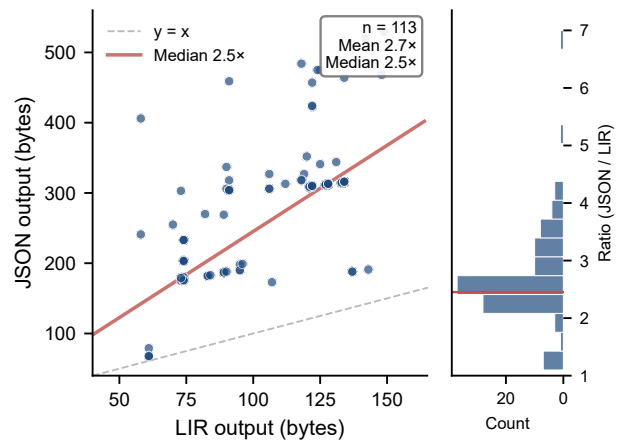


Figure 5: Per-task LIR vs. JSON source size across all 113 tasks (bytes); inset shows the distribution of size ratios. This compares source text only; compilation to bytecode yields the larger ratios reported in Table 2.

7.2.2. On-Device Energy Measurement

To measure on-device energy directly, we instrumented an ESP8266 with a USB power meter (RuiDeng UM25C) on the 5 V supply rail and compared three representations

⁵Each repair round adds approximately 40 output tokens for LIR regeneration and 160 input tokens for error information and the original prompt. Summed over the 13 repaired tasks, the total overhead is approximately 2600 additional tokens across the full set.

executing the same closed-loop task: the LVM bytecode, an on-device JSON interpreter, and hand-written native C. To isolate interpretation cost from physical actuation, relay writes were redirected to a simulated state array and `wait` was skipped; mean power over a 60 s window was divided into per-task energy using measured CPU time.

Table 6: On-device per-task energy (ESP8266 + UM25C, 60 s windows). All three representations execute the identical closed-loop task; differences arise purely from interpretation overhead, not physical actuation.

Representation	Power (mW)	Time (μ s/task)	Energy (μ J/task)	vs. native
Native C (lower bound)	419.3	8.5	3.57	1.0×
Compact bytecode (ours)	418.9	115.7	48.48	13.6×
On-device JSON interp.	417.8	226.3	94.57	26.5×

Mean power is nearly identical across all three groups (~ 418 mW), confirming that the energy difference is attributable to per-task CPU time rather than the physical control action. The UM25C measures whole-board power (including the on-board CH340 USB bridge and regulator quiescent draw); absolute μ J/task values include this common offset, while the *relative* ratios between representations are unaffected.

The bytecode consumes 48.48 μ J/task versus 94.57 μ J/task for the on-device JSON interpreter (Table 6), a 1.95 $\times$ (48.7%) energy reduction. This difference stems from the bytecode’s integer-dispatch execution model, which replaces JSON’s per-key text scanning with direct opcode dispatch. Both interpreted paths sit above the 3.57 μ J/task native-C lower bound (13.6 $\times$ and 26.5 $\times$ respectively). The measurement isolates interpretation cost: with relay writes redirected to a state array and `wait` skipped, it excludes actuator drive and blocking delay, which dominate energy in a deployed task and would dilute the relative gap between interpreters. The figures therefore upper-bound the interpreter’s share of total task energy rather than estimate end-to-end consumption.

The 13.6 $\times$ overhead over native C reflects the cost of interpretation versus compiled code. In deployments where control programs change frequently (e.g., daily irrigation schedules), reflashing the MCU for each update is impractical; the interpreter amortizes its overhead across thousands of executions. At 48.48 μ J/task, the bytecode interpreter closes the gap between update flexibility and battery life that an on-device JSON interpreter widens.

The same encoding choice that shrinks the payload also lowers execution energy, but manifests at a different layer: on the wire it saves bandwidth, on the device it saves CPU cycles.

7.2.3. Token Cost and Cache Dependency

LIR’s token efficiency advantage depends on system-prompt caching, where the system prompt (containing the EBNF grammar, device mapping table, and few-shot examples) is cached and reused across requests. Under caching, the per-request *incremental* cost (output only) is 40.0 tokens for LIR versus 77.5 for JSON (48% fewer) across the 52 three-way semantically equivalent tasks. Without caching, the total token cost is higher for LIR at 201.2 tokens than for JSON

at 160.7; each new device type also invalidates the cached grammar prefix. The payload-size advantages (Table 2) are independent of caching.

7.2.4. Negative Baselines

To further quantify the LIR path’s size advantage over conventional embedded code, we generated Arduino C code on 113 deterministic tasks using an Arduino C prompt of equivalent detail, comparing output sizes.

Arduino C averages 188.1 bytes, 17.3 $\times$ the bytecode (10.9 B) and 1.5 $\times$ even compact JSON (127.18 B). All 113 outputs compiled successfully; the payload cost comes from general-purpose syntax that a specialized IR avoids.

These three gains come from one encoding choice: positional, varint-encoded bytecode in place of self-describing text. It removes repeated keys and delimiters on the wire, cuts the token count for the same control intent, and replaces text scanning with integer dispatch on the device.

7.3. Security: Containment and Ablation

LLM-generated control code is, by construction, untrusted: a model may hallucinate an activation on a device the operator never authorized, emit a malformed instruction stream, or, under prompt injection through the feedback channel (Section 4.1), be steered toward an unintended write. On a microcontroller driving physical relays, any one of these is a safety incident. The central security claim of this work is therefore not that the LLM produces correct programs, but that the LVM confines the LLM’s output to a bounded, capability-respecting envelope before it can touch hardware.

This subsection validates that claim under two complementary regimes. A per-layer ablation (Section 7.3.1) isolates each guard’s independent contribution by disabling it and measuring how containment degrades. An adversarial fuzzing campaign (Section 7.3.2) stresses the combined boundary with 8,000 hostile payloads that never pass through the LLM at all, directly probing the runtime guarantees proved in Section 5. Together they connect the formal result, $UABR = 1$ for unauthorized requests (Corollary 1, from Proposition 3), to measured behavior on both LLM-sourced and synthetically adversarial inputs.

7.3.1. Security Mechanism Ablation

The fully guarded baseline achieves $UABR = 1.000$ (Fig. 6); removing capability gating alone drops it to 0.857, because unauthorized I/O requests that would have been blocked at the gateway now reach the device. Each guard addresses a distinct fault class: capability gating blocks unauthorized devices, the validator rejects malformed bytecode, and the step-limit terminates runaway programs.

7.3.2. Adversarial Bytecode Fuzzing

The ablation above measures *which* guard intercepts a fault, but it does not stress the containment boundary with adversarial inputs. To complement it, we ran a fuzzing campaign against the LVM reference executor (the Python reference implementation, with step-limit $L=1000$ and stack bound 256 enabled to mirror the firmware and the formal model

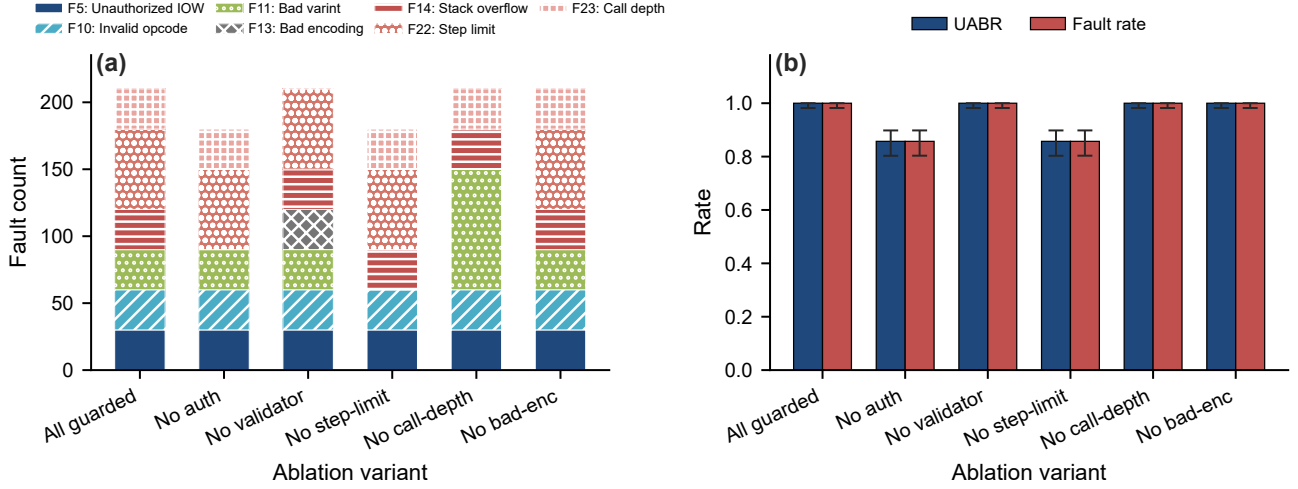


Figure 6: Security mechanism ablation. (a) Fault-type distribution intercepted by each mechanism; (b) UABR vs. fault rate across ablation variants.

Table 7

Adversarial fuzzing containment (8,000 payloads, seed=42, $L=1000$, stack bound 256). “Escaped” = unauthorized side effect or unbounded run.

Attack class	Payloads	Contained	Rate
Unauthorized I/O	1000	1000	1.000
Invalid opcode	1000	1000	1.000
Infinite loop (backward JMP)	1000	1000	1.000
Stack underflow	1000	1000	1.000
Stack overflow	1000	1000	1.000
Jump out-of-bounds	1000	1000	1.000
Truncated variant	1000	1000	1.000
Random bytes	1000	999 [†]	0.999
Overall	8000	7999	0.9999
Escapes	—	0	0.000

[†]The single non-blocked random payload decoded to a benign no-op (a lone HALT-equivalent) that terminated cleanly; it is *not* an escape, since capability gating makes an unauthorized write impossible on a clean exit.

of Section 5). We generate 8,000 malicious or malformed bytecode payloads across eight attack classes (1,000 each): unauthorized I/O (a write to or read from a device never declared via GTWAY), injected invalid opcodes, non-terminating backward jumps, stack underflow, stack overflow, out-of-bounds jump targets, truncated variants, and pure random bytes. For each payload we record the interception stage (static verifier vs. runtime guard), the fault code, and, critically, whether the program *escaped* containment, defined as either performing an unauthorized relay mutation or running unboundedly.

The LVM withstands 8,000 adversarial payloads without observed escapes (Table 7). For the capability-based attack class (unauthorized I/O), UABR remains 1.000; overall containment across all eight attack classes is 0.9999. Interception is roughly evenly split between the static verifier (~50%, dominated by bad-encoding rejections) and the runtime guards (~50%, covering unauthorized-I/O, step-limit, and stack faults). Because containment is independent of payload origin, whether LLM-sourced or synthetically

adversarial, the safety envelope is a property of the LVM runtime rather than of the input source.

These results address a specific concern in LLM-based edge intelligence: that an LLM might hallucinate a malicious or malformed program, or that an attacker might inject adversarial bytecode through a compromised communication channel. The fuzzing campaign simulates both scenarios by generating 8,000 payloads that no LLM would intentionally produce (invalid opcodes, truncated encodings, infinite loops, stack violations) and confirming that the LVM catches every one before it can reach hardware. The zero-escape result means that the security guarantee does not depend on the LLM being well-behaved; it is a property of the runtime itself.

Theoretical vs. empirical bounds. Proposition 3 guarantees termination within L steps. For the 113-task set, average execution length is 5.8 steps with $L=1000$, utilizing 0.58% of the step budget. The theoretical bound is thus roughly 170× the empirical average, indicating substantial safety margin.

7.3.3. System-Level Pipeline Ablation

Only the full LIR configuration satisfies all four properties at once (Table 8); removing any single layer forfeits one of them. Arduino C succeeds on every task but emits 188.1 bytes per program and provides no execution bounds. JSON reaches 0.965 at 106.7 bytes and gains bounded execution only when paired with an MCU-side interpreter, and even then without capability gating. LIR matches the best generation reliability at a tenth of the payload (10.9 bytes) while retaining both bounded execution and grammar guarantees. Dropping the grammar layer leaves reliability intact at temperature zero but, as the temperature sweep below shows, sacrifices the stability that holds generation together under sampling.

As shown in Table 8, replacing LIR with Arduino C preserves correctness (TSR = 1.000) but produces 17.3× larger payloads and provides no bounded execution guarantees. Replacing LIR with JSON + zlib reduces payload to 106.7 B but drops TSR to 0.965 and still lacks bounded

Table 8

System-level pipeline ablation (113 tasks, temperature=,0.0).

Configuration	TSR	Bytes	Bounded	Grammar
Arduino C	1.000	188.1	×	×
JSON + zlib	0.965	106.7	×	×
JSON + zlib + interpreter [†]	0.965	106.7	✓ [§]	×
LIR (no grammar)	1.000 [†]	10.9	✓	×
LIR (full)	1.000	10.9	✓	✓

[†]At temperature=,0.0; degrades to 0.847 at temperature=,1.0 without grammar constraints (Table 9). [‡]JSON output parsed by a minimal MCU-side JSON interpreter with step-limiting; bounded execution achieved through interpreter-level enforcement rather than formal VM guarantees. [§]Interpreter-level bounded execution only; no capability gating or formal safety proofs.

execution. The full LIR pipeline achieves TSR = 1.000 with 9.8× compression over optimized JSON (13.8× over raw-LLM zlib-JSON) and bounded execution with formal guarantees. Grammar-constrained decoding becomes critical at non-zero temperatures: without it, TSR degrades to 0.847 at temperature = 1.0. Empirically measured best-of- K ($K=3$) recovers reliability to 0.982 (Table 11).

These ablation and fuzzing results establish that the LVM contains adversarial output and that each guard contributes independently to that containment.

7.4. Reliability: Generation Stability

This subsection answers Q3 by measuring how reliably the pipeline produces correct programs: under closed-loop retry, across the sampling-temperature range, over a large random task space, and on tasks drawn from real-world IoT deployments.

7.4.1. Closed-Loop Control

Retry raises SCR under both paths (Fig. 7(a)). Under the same retry strategy the LIR path achieves lower RTT than the JSON path, so the convergence benefit derives from the retry mechanism itself rather than from which layer carries the closed-loop semantics.

7.4.2. SAFE Shunting and Fault Prior Sweep

Fig. 7(a) shows that retry raises the Safe Convergence Rate under both LIR and JSON paths, with LIR achieving lower RTT. Across the 10%–90% fault-prior range, the guarded group maintains stable shunting performance while the no-guard group cannot reliably identify fault paths (Fig. 7(b)).

7.4.3. Temperature Sweep

The closed-loop results above use temperature = 0.0 (deterministic generation). To understand how the pipeline behaves under stochastic sampling, which is necessary for creative task variants and multi-candidate selection, we sweep the temperature from 0.0 to 1.0. The pipeline attains a pass rate of 1.000 at temperature = 0.0 and degrades smoothly to 0.847 at temperature = 1.0 (Table 9), with IR validity staying above 0.91 throughout. The decline is driven by semantic deviation under sampling rather than format breakdown, which motivates the best-of- K recovery measured next (Section 7.4.6).

Table 9

Temperature sweep results (6 temperature points, 339 trials each, 95% Wilson CIs). Each trial is an independent LLM generation on one of 113 tasks.

Temp.	TSR (95% Wilson CI)	IR Valid
0.0 (deterministic)	1.000 [0.989, 1.000]	1.000
0.2	0.985 [0.966, 0.994]	0.988
0.4	0.941 [0.911, 0.961]	0.971
0.6	0.882 [0.843, 0.912]	0.950
0.8	0.858 [0.817, 0.891]	0.914
1.0	0.847 [0.804, 0.881]	0.926

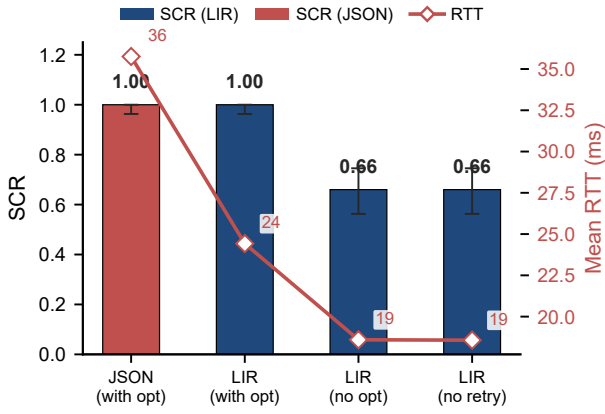
At temperature = 0.0, the pipeline is fully reliable for safety-critical tasks. At temperature = 1.0, the pass rate drops to 0.847; best-of- K ($K=3$) sampling recovers it to 0.982 (Section 7.4.6), at the cost of additional inference calls. IR validity stays above 0.91 even at the highest temperature, indicating that the decline is driven by semantic errors (wrong device, wrong value) rather than format breakdown, exactly the class of errors that capability gating and readback are designed to catch.

7.4.4. Random Task Coverage

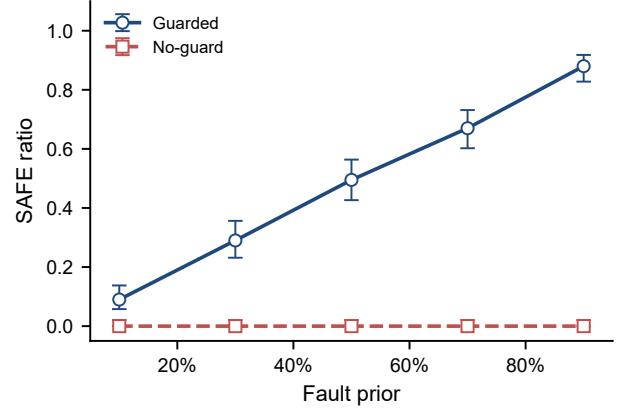
To measure how reliably the LIR generation chain covers a large space of parameter combinations, we automatically generated 600 tasks based on 12 categories of parameterized templates (50 per category, with random device combinations and parameters), evaluated on llama3.1:8b (temperature=,0.0) and additionally on deepseek-v4-pro (commercial API, temperature=,0.0) to assess cross-model consistency. Because these tasks are drawn from the same template distribution as the development set, this experiment measures robustness to parameter variation within the supported task space, not out-of-distribution generalization (the latter is assessed on the real-world task set of Section 7.4.5). The aggregate results are reported here; the per-category breakdown is in Appendix C.2, Table 16.

On the 600-task random set, llama3.1:8b achieves an IR validity rate of 0.997, compile pass rate of 0.997, and task success rate of 0.970 (95% Wilson CI: [0.953, 0.981]). The commercial model deepseek-v4-pro reaches 0.987 on the same set (592/600), with 6 of 12 categories reaching 1.000. The consistent improvement across models confirms that LIR's advantage is representation-level: stronger models reduce format errors but the payload-compactness and execution-safety benefits hold regardless of which model generates the LIR.

7 of 12 categories achieved 100% (Appendix C.2, Table 16); the hardest were the write-then-wait-then-read and write-then-wait-then-multi-read patterns (both 90%) and long sequences (96%), where the dominant failure mode is semantic deviation in multi-step long sequences. The LIR generation chain thus remains reliable across task types and parameter combinations within the supported task space.



(a) SCR: LIR vs. JSON, with/without retry.



(b) SAFE shunting vs. fault prior.

Figure 7: Closed-loop performance. (a) Retry raises SCR under both LIR and JSON paths. (b) The guarded group maintains stable SAFE shunting across fault priors.

Table 10

Real-world IoT task evaluation ($n=29$, temperature= 0.0): overall and sequential aggregates for LIR v1 vs v4 prompt and the JSON baseline. The per-category retry/readback breakdown is in Appendix C.2, Table 17.

	LIR v1	LIR v4	JSON
Overall			
IR valid rate	0.724	0.828	1.000
Task success rate	0.586	0.793	0.862
Sequential tasks (19)			
All sequential	17/19	17/19	18/19

7.4.5. Real-World IoT Task Evaluation

To evaluate LIR generation on tasks derived from real-world IoT deployments, we constructed a 29-task set by extracting control patterns from four open-source IoT platforms: Home Assistant automation rules, Tasmota device rules, ESPHome YAML configurations, and Arduino/ESP8266 DIY projects. These tasks cover 11 control pattern categories including threshold-based actuation, timed pulses, emergency shutoffs, multi-relay sequencing, and retry-with-verification loops. Unlike the synthetic tasks in the random-task coverage and Arduino C baseline experiments, these tasks reflect control logic that engineers actually deploy on ESP8266/ESP32 hardware.

Table 10 shows results under three configurations. Under the v1 prompt, 17 of 19 sequential tasks succeed (0.895) while all closed-loop tasks fail (0.000); the v4 prompt (adding 4 few-shot examples for retry/readback patterns) lifts the closed-loop categories to 0.600–1.000 and the overall rate from 0.586 to 0.793. The closed-loop subset (10 tasks) achieves TSR= 0.600 with 95% Wilson CI [0.313, 0.832]. The JSON baseline reaches 0.862 overall, above LIR v4’s 0.793, at 4.5 $\times$ the payload (zlib-JSON 110.3 B vs. LIR bytecode 24.5 B, averaged over the 29-task set) and without formal bounded execution guarantees. The 4 LIR v4 failures are all LIR_PARSE_ERROR cases where the LLM generates syntactically invalid retry/readback compositions.

Failure root cause analysis. The 4 LIR v4 closed-loop failures split into two categories: (1) *Invalid retry nesting* (2 failures): incorrect brace nesting or missing readback keywords; (2) *Unsupported keyword injection* (2 failures): the LLM generates keywords outside the LIR grammar (e.g., while, if). All 4 are syntax-level errors; no semantic-level failures (compilation success but incorrect device state) were observed. The failure taxonomy across all task sets is summarized in Appendix C.2.

7.4.6. Grammar-Constrained Decoding

To quantify the improvement of grammar-constrained decoding on sampling-mode reliability, we evaluate two complementary approaches: (i) *post-hoc grammar filtering*: generate $K = 5$ candidates per task at temperature= 1.0 , filter using the LIR EBNF grammar, and accept the first valid candidate; (ii) *integrated grammar-constrained decoding*: use a CFG-based decoder (e.g., Outlines [42]) to enforce the LIR EBNF grammar during generation, guaranteeing syntactic validity of every output. Table 11 gives the results.

Grammar filtering itself provides a +3.5% improvement (0.841 $\rightarrow$ 0.876); the improvement is limited because the grammar validity rate has already reached 0.950, with the main bottleneck not in format but in semantics. The unconstrained first-attempt rate here (0.841, a single $K=1$ pass over the 113 tasks at temperature = 1.0) is the same quantity as the 0.847 reported in the temperature sweep (Table 9); the small difference reflects sampling noise, since the sweep value is a mean over three independent passes ($n=339$) while this is one pass. To validate this proxy, we independently reran the experiment generating $K=5$ candidates per task at temperature = 1.0 and evaluated *every* candidate to end-of-pipeline task success (via an independent best-of- K evaluation harness). The resulting best-of- K curve (0.823, 0.956, 0.982, 0.991, 1.000 for $K=1 \dots 5$; grammar-valid rate 0.970) closely corroborates the post-hoc proxy: a single resample ($K=2$) already lifts reliability from 0.823 to 0.956, and $K=3$ reaches 0.982, with full coverage by $K=5$. The grammar-filtered and unconstrained curves coincide here because

Table 11

Grammar-constrained decoding results (temperature=1.0, $K = 5$, 113 tasks): measured post-hoc filtering and best-of- K task success.

Metric	Value
Unconstrained success (first)	0.841
Grammar-filtered success (first valid)	0.876
Avg. grammar validity rate	0.950
Grammar-filtered improvement	+0.035
K-best analysis (analytic proxy from validity rate)[†]	
$K = 1$	0.841
$K = 2$	0.975
$K = 3$	0.996
$K = 5$	~1.000
Empirically measured best-of-K (this work, $n=113$)	
$K = 1$	0.823
$K = 2$	0.956
$K = 3$	0.982
$K = 4$	0.991
$K = 5$	1.000

[†]Only $K=1$ (0.841) and the grammar-filtered success (0.876) are directly measured (result/GEN/grammar_constrained.csv); the $K \geq 2$ entries are projected from the 0.950 grammar-validity rate as a filtering proxy. The empirically measured best-of- K block below (result/GEN/success_at_k.csv) replaces this proxy with per-candidate end-to-end evaluation and is the figure cited in the text.

at this grammar-valid rate (0.970) the first grammar-valid candidate is almost always available within the budget. These results show that multi-sampling is a viable solution to the sampling-mode reliability problem, directly addressing the discussion in Section 8.1. An integrated CFG decoder (e.g., Outlines [42]) that enforces the grammar during generation would eliminate format errors at $K=1$; we did not measure this configuration because Ollama’s API does not expose grammar-constrained decoding.

A deployment can thus choose its reliability–cost trade-off along this curve: at $K=1$ (single attempt, lowest cost) the pass rate is 0.823, which may suffice for non-critical tasks; at $K=3$ (three attempts, moderate cost) it reaches 0.982, suitable for most field deployments; at $K=5$ it reaches 1.000 on this task set. The cost of each additional attempt is one extra LLM inference (roughly 40 output tokens for LIR regeneration), which is small compared to the cost of deploying a faulty program to a physical device.

To verify the LIR compiler’s support for control-flow constructs, we additionally constructed 15 tasks covering if/else conditional branching, repeat N times loops, and their nested combinations. Appendix C.2 Table 13 gives llama3.1:8b’s generation results on this task set. Two of the 15 fail: one adds an extra set `relay2 = 0` inside the repeat block (semantic deviation), the other uses the unsupported and keyword and `!relay1` syntax (parse error); both are LLM generation deviations that the grammar constraints correctly block rather than compiler defects. To further verify closed-loop semantic generation, we expanded the closed-loop set from 16 to 96 tasks across 12 subcategories; Appendix C.2 Table 13 reports the results.

Detailed generation pipeline, token comparison, and semantic pass-rate results are consolidated in Appendix A (Table 12); Fig. 8 summarizes the pipeline and token comparison visually.

7.5. Portability: Cross-Platform Validation

To validate LVM’s platform independence, we ported the firmware to STM32F103C6T6 (ARM Cortex-M3, 32 KB flash, 10 KB RAM) using the STM32duino board package. The port required fewer than 50 lines of code changed, limited to I/O adapter functions (`read_sensor()`, `write_actuator()`) and pin definitions; the LVM core (stack machine, variant decoder, capability gating, step-limit) remained unchanged.

Build results. The STM32 firmware compiled to 21,212 bytes flash (64%) and 2,740 bytes RAM (26%), fitting comfortably within the STM32F103C6’s 32 KB flash and 10 KB SRAM (Table 4).

Functional verification. We verified correct execution of three representative bytecode programs via CH340 serial communication at 115200 baud:

1. HALT: single-instruction termination, OK (1 step).
2. GTWAY 5; LIT 1; IOW 5; HALT: capability-gated relay actuation, OK (4 steps, relay PB0 activated).
3. GTWAY 1; IOR 1; HALT: capability-gated sensor read, OK (3 steps, water sensor ADC value returned).

Each program returned correct OK responses with expected step counts. The capability gating mechanism correctly enforced device authorization on both platforms. These results confirm that LVM’s safety guarantees (Propositions 1–3) hold on ARM Cortex-M, not just on Xtensa.

Limitations. STM32 validation covers bytecode execution only; the end-to-end LLM→LIR→bytecode pipeline was not re-run on STM32 (the LLM generation and compilation steps are platform-independent). DHT11 temperature/humidity sensor support was compiled into the firmware but the sensor was not physically connected during validation.

In summary, the evaluation answers the four questions in turn. For *efficiency* (Q1), compiling LIR to bytecode yields a 9.8× smaller payload than an optimized JSON baseline (13.8× against raw-LLM zlib-JSON) and reduces on-device interpretation energy by 48.7% versus a JSON interpreter, with output tokens 48% lower under prompt caching. For *security* (Q2), the LVM blocks all unauthorized access requests in our experiments (UABR = 1.000) and withstands 8,000 adversarial payloads without observed escapes, each guard contributing an independent layer. For *reliability* (Q3), the pipeline attains a 1.000 pass rate at temperature = 0.0 on the 113-task set and 0.970 on 600 random tasks, with best-of- K ($K=3$) restoring temperature = 1.0 reliability to 0.982 and real-world sequential IoT tasks reaching 0.895. For *portability* (Q4), identical bytecode executes correctly on both ESP8266 (Xtensa) and STM32F103 (ARM Cortex-M3) with fewer than 50 lines of I/O-adaptor changes.

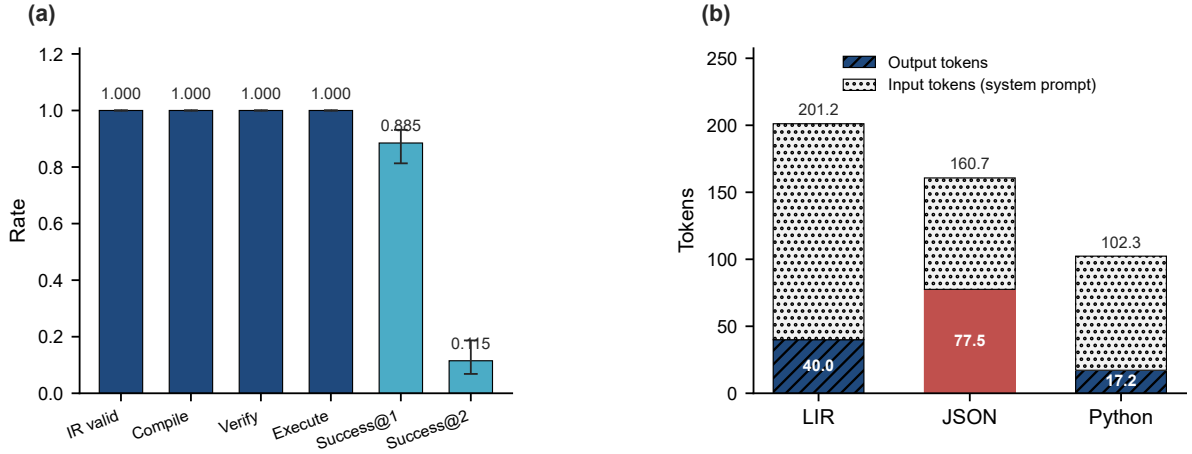


Figure 8: Generation pipeline and token cost. Panel (a): pipeline and repair-loop results on 113 deterministic tasks. Panel (b): token and output-size comparison on 52 three-way semantically equivalent tasks.

8. Discussion

8.1. Design Trade-offs and Positioning

LIR's syntactic strictness trades generative tolerance for compactness and safety. On closed-loop tasks, LIR v4 reaches a success rate of 0.600, below JSON's 0.700, because the EBNF rejects invalid retry/readback compositions that JSON's permissive parser accepts. This strictness enables 4.5× payload reduction and bounded-execution guarantees. On token cost without caching, LIR consumes 201.2 tokens against JSON's 160.7. LIR's efficiency advantage requires system-prompt caching to amortize the longer grammar prefix. Both trade-offs are architectural: the two-layer design optimizes for the cloud-to-MCU deployment scenario where caching infrastructure is available and payload size dominates total cost.

LIR serves as a structured tool definition for embedded devices, compatible with agent frameworks like LangChain [11]. Whereas container- and kernel-level isolation such as Docker/gVisor [3] targets server-class hosts, the LVM provides bounded execution at the MCU level, where those runtimes do not fit. The two operate at different points of the deployment stack rather than as direct substitutes. Among embedded runtimes, WAMR's 15–60 KB RAM footprint exceeds the 10–80 KB SRAM of typical edge MCUs; LVM's ~2 KB fits within the same budget, leaving room for application logic and sensor buffers.

8.2. Cross-Platform Validation Scope

End-to-end LLM generation experiments were conducted on ESP8266; compiled bytecode execution was confirmed on STM32F103 with fewer than 50 lines of I/O-adaptor code changed. Verified task types cover 7 deterministic, 12 closed-loop, 3 control-flow, 12 random, and 11 real-world IoT patterns (Appendix C.5); unverified types include nested closed-loop and strong real-time control. Full pipeline re-validation on additional ARM Cortex-M variants is future work.

8.3. Closed-Loop Prompt Engineering

Closed-loop patterns (retry/readback) reach a success rate of 0.600 with few-shot prompting on the 10-task real-world subset. The v1→v4 improvement is concentrated in closed-loop patterns (sequential tasks achieve 0.895 under both prompts), indicating that LIR's core syntax is inherently LLM-friendly but complex compositions benefit from explicit examples. These examples are device-independent and transfer across deployments; integrating them into a production prompt template is straightforward but has not been evaluated at scale.

8.4. Grammar Scalability and Model Dependency

LIR's current grammar covers 7 statement types and 5 device names. Scaling to production IoT deployments (50+ device types) would expand the system prompt and may reduce LLM generation accuracy. The 113-task set is hand-crafted; this is mitigated by the 600-task random set and the 29-task real-world set (extracted from open-source IoT platforms), but generation reliability at production scale remains untested. LLM generation reliability depends on instruction-following capability: llama3.1:8b and deepseek-r1:8b both achieve TSR = 1.000, while qwen3:8b achieves 0.903. This gap is consistent with a grammar-constrained decoding fix, not an LIR redesign, and integrated CFG decoders [42] remain future work. Validation on larger models (70B+) is also future work. TSR measures device-state correctness, not path optimality; the security guarantees (capability gating, step limits) hold regardless of which model generates the LIR, as shown by the formal analysis (Section 5) and the adversarial fuzzing campaign (Section 7.3.2).

9. Related Work

9.1. Formal Methods for Embedded Safety

Formal verification has been applied to systems software at several levels: CompCert [24] verifies a C compiler's semantic preservation, seL4 [23] proves microkernel functional correctness, Native Client [43] verifies x86 binaries

statically, and eBPF [16] runs an in-kernel verifier that bounds termination and memory safety of user programs [36]. Their verification targets (general C semantics, full kernels, richly typed bytecode) span far larger semantic surfaces than the LVM's instruction subset, so their proofs require correspondingly heavier machinery. The LVM restricts itself to a 14-instruction subset of fixed-step control operations (Table 5), whose safety properties reduce to case analysis over a finite rule set, keeping its proofs (Propositions 1–3) small enough to state and check within the paper. Its capability gating is a minimal instantiation of capability-based security, conceptually akin to seccomp-BPF [27] at the OS level or CHERI [41] in hardware, but realized at the bytecode-interpreter level within 2 KB of RAM. This makes it applicable to bare-metal MCUs with no OS or hardware support. Related embedded attestation work [40, 17, 37] operates at the binary level rather than the instruction-set level.

9.2. LLM-Based Code Generation for Edge

Code-as-Policies [26] generates Python policies for robot arms, and RT-2 [7] fine-tunes vision-language models for action output. Both target desktop-class robots. Tool-use frameworks [35, 32] broaden how LLMs invoke APIs but do not address on-device execution safety. Esposito et al. [15] identify code-correctness and resource constraints as the two central challenges for LLM-generated embedded code, yet existing mitigation strategies (grammar-constrained decoding for format validity, sandbox runtimes for execution isolation) address only one side each. To our knowledge, LIR + LVM is the first to integrate a generation-targeted IR with formal bounded execution on resource-constrained MCUs.

9.3. IoT Stack Integration

LIR-generated bytecode is complementary to, not competing with, the surrounding IoT stack. It can travel over MQTT/CoAP [31, 39] and execute on bare metal or atop an RTOS [38, 4, 33]. The compact binary encoding uses varint/zigzag formats compatible with CBOR/MessagePack [6, 30] serialization. Such bounded executors are complementary to device operating systems and scripting runtimes [5, 1]: where MQTT/CoAP stacks address transport reliability and RTOS runtimes address task scheduling, the LVM provides the execution safety layer that these stacks lack for LLM-generated control in edge intelligence deployments.

10. Conclusion

We have described LIR + LVM, a two-layer framework that translates natural-language hardware control instructions into safely executable MCU programs. The framework decouples generation-friendly LIR from execution-friendly bytecode via a deterministic compiler, with LVM providing bounded execution guarantees for the 14-instruction set including readback/retry (Propositions 1–3, Appendix C.1). Existing approaches address either generation friendliness or execution compactness in isolation; LIR + LVM addresses both simultaneously.

The framework achieves a 9.8× payload reduction when cloud-compiled to bytecode against an optimized JSON baseline (13.8× against raw-LLM zlib-JSON), with UABR = 1.000 in adversarial fuzzing. For sequential control tasks, the dominant pattern in real-world IoT, the pipeline achieves full pass rate on the 113-task deterministic set and 0.970 on 600 random tasks. Under temperature = 1.0 sampling, best-of- K ($K=3$) restores reliability to 0.982. On 19 real-world sequential IoT tasks, 17 achieve success; on the full 29-task set including closed-loop patterns, LIR v4 achieves 0.793 overall. Cross-platform validation confirms that identical bytecode executes correctly on both ESP8266 (Xtensa) and STM32F103 (ARM Cortex-M3) with fewer than 50 lines of code changed.

Future work includes integrating a production-grade CFG decoder, extending LVM validation to additional ARM Cortex-M variants, and enriching the LIR grammar for richer control-flow constructs. By decoupling generation-friendly LIR from execution-friendly bytecode, LIR + LVM fills a gap in the edge intelligence design space: compact, formally bounded, and cross-platform MCU execution of untrusted LLM output.

Artifact Availability. All code, datasets, firmware, and compiler source are open-sourced at <https://github.com/MYJ-GOD/lir-lvm>.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used Claude (Anthropic) in order to improve the language and readability of the manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Data Availability Statement

All code, datasets, firmware, and compiler source are publicly available at <https://github.com/MYJ-GOD/lir-lvm>.

Declaration of Competing Interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the Youth Fund Project of North China Institute of Aerospace Engineering (No. KY-2024-05), the Science and Technology Support Program of Langfang (No. 2024011028), and the Scientific Research Project of Hebei Provincial Department of Education (No. BJ2025339).

CRedit authorship contribution statement

Yingjie Ma: Conceptualization, Methodology, Software, Writing – review & editing. **Mengjia Ma:** Data curation, Validation, Writing – review & editing. **Sheng Zhao:** Validation, Investigation, Writing – review & editing. **Peng Liu:** Supervision, Conceptualization, Funding acquisition, Writing – original draft, Writing – review & editing.

Author Vitae

Yingjie Ma is a student at the NCIAE, Langfang, China. His research interests include embedded systems, IoT architectures, and LLM-based code generation for resource-constrained devices.

Mengjia Ma is a student at the NCIAE, Langfang, China. Her research interests include large language models, AI-driven code generation, and intelligent edge computing.

Sheng Zhao is a student at the NCIAE, Langfang, China. His research interests include microcontroller firmware, real-time systems, and hardware-software co-design.

Peng Liu is a faculty member at the NCIAE, Langfang, China. His research interests include embedded systems, IoT architectures, and edge intelligence. He is the corresponding author of this paper.

A. Supplementary Tables

1. Compression ratio and transmission latency (Section 7.2.1)
2. Guarded / no-guard safety ablation (Section 7.3.1)
3. Validator ablation (Section 7.3.1)
4. Single-mechanism orthogonal ablation (Section 7.3.1)
5. SCR closed-loop experiment (Section 7.4.1)
6. Real JSON fair baseline (Section 7.4.1)
7. Fault prior sweep (Section 7.4.2)
8. Throughput / memory proxy

Two pass-rate columns. The LIR semantic pass rate (0.867) and the end-to-end TSR = 1.000 of Section 7.1 measure different conditions. The 0.867 figure is the single-shot rate at which a minimal (non-few-shot) prompt yields a semantically correct program on the first attempt, scoring LIR, JSON, and Python on identical footing for raw format-friendliness. TSR = 1.000 is the full-pipeline outcome on the 113-task set under the deployed few-shot prompt with the bounded repair loop (Section 7.1.1).

Python (detailed). Python (detailed) also reaches TSR = 1.000 (Table 12, semantic pass-rate block) with fewer output tokens (19.96 vs. 38.09 for LIR). It requires a host-side Python interpreter (30–300 KB RAM) for on-device execution, provides no bounded execution guarantees, and cannot be compiled to bytecode for bandwidth-constrained links. LIR’s payload reduction (9.8× over optimized JSON) and formal

Table 12

Consolidated 113-task results (temperature,=,0.0): generation pipeline, repair loop, token cost (input/output/total), and single-shot semantic pass rates. Token totals = system prompt + output.

Generation pipeline			
IR / Compile / Verify / Exec / Task		1.000	[0.967, 1.000]
Repair loop			
Success@1		100/113 = 0.885	[0.813, 0.932]
Success@2		13/113 = 0.115	[0.068, 0.187]
Token cost			
Format	Input	Output	Total
LIR	159.85	38.09	197.94
JSON	81.85	78.14	159.99
Python	83.85	19.96	103.81
Semantic pass rate (single-shot)			
Format	Passed	Rate	95% CI
LIR	98/113	0.867	[0.791, 0.919]
JSON	90/113	0.796	[0.712, 0.862]
Python (basic)	78/113	0.690	[0.599, 0.769]
Python (detailed)	113/113	1.000	[0.967, 1.000]

bounded execution are orthogonal to TSR and hold regardless of which format scores higher under detailed prompting.

B. Complete Proofs

Throughout, let $\sigma_0 = \langle p, 0, \epsilon, dev_0, cap_0, 0, L, \perp, r_0 \rangle$ be the initial configuration with program counter at zero, empty stack, initial device state, declared capabilities, step counter at zero, no fault, and initial retry counter r_0 (set by the compiler before entering any retry block; $r_0 = 0$ if no retry is active).

PROOF (PROOF OF PROPOSITION 1 (DETERMINISM)). Each transition rule is uniquely determined by: (i) the instruction $p[pc]$, (ii) the step-counter condition $k < L$ vs. $k \geq L$, (iii) the capability membership $d \in cap$ vs. $d \notin cap$, and (iv) the stack contents. These conditions are mutually exclusive and exhaustive for each instruction: exactly one rule applies per configuration, so the successor is unique.

PROOF (PROOF OF PROPOSITION 2 (PROGRESS)). If $pc \geq |p|$, the configuration is terminal by definition (iv). Otherwise $p[pc]$ is some instruction I . We proceed by cases: (1) $I = \text{HALT}$: terminal by rule HALT. (2) $I \in \{\text{LIT}, \text{GTWAY}, \text{WAIT}\}$: no stack precondition, transition always applies. (3) $I \in \{\text{IOW}, \text{IOR}\}$: if $d \in cap$, rule IOW-OK/IOR-OK applies; if $d \notin cap$, rule IOW-CAPF/IOR-CAPF applies. (4) $I \in \{\text{EQ}, \text{SUB}\}$: if $|s| \geq 2$, the rule applies; otherwise POPF. (5) $I \in \{\text{JZ}, \text{DRP}\}$: if $|s| \geq 1$, the rule applies; otherwise POPF. (6) $I = \text{DUP}$: if $|s| \geq 1$, DUP; otherwise POPF. (7) $I = \text{JMP}$: no precondition, JMP always applies. (8) $I \notin \text{Opcodes}$: OPCF. In every case, at least one rule applies.

PROOF (PROOF OF PROPOSITION 3 (BOUNDED EXECUTION)).

(1) Termination. Every transition rule increments k by exactly 1, including JMP and JZ (which modify pc but still increment k by 1). Thus for any step $\sigma \xrightarrow{1} \sigma'$, we have $\sigma'.k = \sigma.k + 1$. Starting from $\sigma_0.k = 0$, after at most L steps we reach σ_n with $\sigma_n.k \geq L$, at which point STEPGUARD triggers (producing a terminal configuration with $f = \text{STEP}$) unless an earlier terminal configuration

was reached. Therefore every execution reaches a terminal configuration in at most L steps.

(2) Capability safety. The first instruction $\text{IOW } d$ or $\text{IOR } d$ with $d \notin \text{cap}$ triggers IOW-CAPF or IOR-CAPF , setting $f = \text{CAP}$.

(3) Opcode safety. The first invalid opcode triggers OPCF , setting $f = \text{OPC}$.

B.1. RETRY Compilation Strategy

The compiler lowers each retry n times $\{B\}$ construct into a sequence that initializes the retry counter r before entering the loop body. Specifically, the compiler emits:

1. $\text{LIT } n$, push the retry count onto the stack;
2. Store the value into the retry counter r (via an internal SET_RETRY operation that sets $r \leftarrow n$);
3. The loop body B ;
4. $\text{RETRY } n \ o$, where o is the jump offset back to the start of B .

At each RETRY execution, the rule RETRY-LOOP decrements r ($r' = r - 1$) and jumps if $r > 0$; when $r = 0$, RETRY-EXHAUST falls through. This ensures that the loop body executes exactly n times, and the retry counter r is strictly decreasing, guaranteeing bounded termination.

For nested retry constructs, the compiler emits separate counter initializations for each level. The current implementation does not support runtime-variable retry counts (e.g., $\text{retry_sensor_value}$ times $\{\dots\}$); the count n must be a compile-time constant. This restriction is part of the LIR v0.1 expression power boundary (Section 3.6).

C. Prompts and Instruction Set

C.1. Complete Transition Rules

The single-step transition relation $\xrightarrow{1}$ is defined by rules for each instruction (LIT , GTWAY , IOW , IOR , WAIT , HALT , EQ , JZ , JMP , DUP , SUB , DRP , READBACK , RETRY), plus stack underflow, invalid opcode, and step-limit guard rules. Each rule specifies preconditions on the instruction, step counter, capability set, and stack contents, and defines the successor configuration. The complete set of 17 transition rules with the revised configuration $\sigma = \langle p, pc, s, dev, cap, k, L, f, r \rangle$ is defined as follows. In each rule, the default successor increments both the program counter and step counter: $\sigma' = \langle p, pc+1, s', dev', cap, k+1, L, f', r \rangle$, unless otherwise specified for JMP , JZ , and RETRY (which modify pc by the offset o while k still increments by 1):

Step-limit guard. For every instruction, if $k \geq L$, the machine transitions to a fault state:

$$\frac{k \geq L}{\langle p, pc, s, dev, cap, k, L, \perp, r \rangle} \quad [\text{STEPGUARD}]$$

$$\xrightarrow{1} \langle p, pc+1, s, dev, cap, k+1, L, \text{STEP}, r \rangle$$

LIT v .

$$\frac{p[pc] = \text{LIT } v \quad k < L}{\langle p, pc, s, dev, cap, k, L, \perp, r \rangle} \quad [\text{LIT}]$$

$$\xrightarrow{1} \langle p, pc+1, s \cdot v, dev, cap, k+1, L, \perp, r \rangle$$

GTWAY d .

$$\frac{p[pc] = \text{GTWAY } d \quad k < L}{\langle p, pc, s, dev, cap, k, L, \perp, r \rangle} \quad [\text{GTWAY}]$$

$$\xrightarrow{1} \langle p, pc+1, s, dev, cap \cup \{d\}, k+1, L, \perp, r \rangle$$

IOW d . If $d \notin \text{cap}$, trigger capability fault; otherwise pop and write:

$$\frac{p[pc] = \text{IOW } d \quad k < L \quad d \notin \text{cap}}{\langle p, pc, s, dev, cap, k, L, \perp, r \rangle} \quad [\text{IOW-CAPF}]$$

$$\xrightarrow{1} \langle p, pc+1, s, dev, cap, k+1, L, \text{CAP}, r \rangle$$

$$\frac{p[pc] = \text{IOW } d \quad k < L \quad d \in \text{cap} \quad s = s' \cdot v}{\langle p, pc, s, dev, cap, k, L, \perp, r \rangle} \quad [\text{IOW-OK}]$$

$$\xrightarrow{1} \langle p, pc+1, s', dev[d \mapsto v], cap, k+1, L, \perp, r \rangle$$

IOR d .

$$\frac{p[pc] = \text{IOR } d \quad k < L \quad d \notin \text{cap}}{\langle p, pc, s, dev, cap, k, L, \perp, r \rangle} \quad [\text{IOR-CAPF}]$$

$$\xrightarrow{1} \langle p, pc+1, s, dev, cap, k+1, L, \text{CAP}, r \rangle$$

$$\frac{p[pc] = \text{IOR } d \quad k < L \quad d \in \text{cap}}{\langle p, pc, s, dev, cap, k, L, \perp, r \rangle} \quad [\text{IOR-OK}]$$

$$\xrightarrow{1} \langle p, pc+1, s \cdot dev(d), dev, cap, k+1, L, \perp, r \rangle$$

WAIT m .

$$\frac{p[pc] = \text{WAIT } m \quad k < L}{\langle p, pc, s, dev, cap, k, L, \perp, r \rangle} \quad [\text{WAIT}]$$

$$\xrightarrow{1} \langle p, pc+1, s, dev, cap, k+1, L, \perp, r \rangle$$

HALT.

$$\frac{p[pc] = \text{HALT} \quad k < L}{\langle p, pc, s, dev, cap, k, L, \perp, r \rangle \text{ is terminal.}} \quad [\text{HALT}]$$

EQ.

$$\frac{p[pc] = \text{EQ} \quad k < L \quad s = s'' \cdot a \cdot b}{\langle p, pc, s, dev, cap, k, L, \perp, r \rangle} \quad [\text{EQ}]$$

$$\xrightarrow{1} \langle p, pc+1, s'' \cdot [a = b], dev, cap, k+1, L, \perp, r \rangle$$

where $[a = b]$ is 1 if $a = b$ and 0 otherwise.

JZ o .

$$\frac{p[pc] = \text{JZ } o \quad k < L \quad s = s' \cdot 0}{\langle p, pc, s, dev, cap, k, L, \perp, r \rangle} \quad [\text{JZ-TAKEN}]$$

$$\xrightarrow{1} \langle p, pc+o, s', dev, cap, k+1, L, \perp, r \rangle$$

$$\frac{p[pc] = \text{JZ } o \quad k < L \quad s = s' \cdot v \quad v \neq 0}{\langle p, pc, s, dev, cap, k, L, \perp, r \rangle} \quad [\text{JZ-NOT TAKEN}]$$

$$\xrightarrow{1} \langle p, pc+1, s', dev, cap, k+1, L, \perp, r \rangle$$

JMP o .

$$\frac{p[pc] = \text{JMP } o \quad k < L}{\langle p, pc, s, dev, cap, k, L, \perp, r \rangle} \quad [\text{JMP}]$$

$$\xrightarrow{1} \langle p, pc+o, s, dev, cap, k+1, L, \perp, r \rangle$$

DUP.

$$\frac{p[pc] = \text{DUP} \quad k < L \quad s = s' \cdot v}{\langle p, pc, s, dev, cap, k, L, \perp, r \rangle} \quad [\text{DUP}]$$

$$\xrightarrow{1} \langle p, pc+1, s' \cdot v \cdot v, dev, cap, k+1, L, \perp, r \rangle$$

SUB.

$$\frac{p[pc] = \text{SUB} \quad k < L \quad s = s'' \cdot a \cdot b}{\langle p, pc, s, dev, cap, k, L, \perp, r \rangle} \quad [\text{SUB}]$$

$$\xrightarrow{1} \langle p, pc+1, s'' \cdot (b - a), dev, cap, k+1, L, \perp, r \rangle$$

DRP.

$$\frac{p[pc] = \text{DRP} \quad k < L \quad s = s' \cdot v}{\langle p, pc, s, dev, cap, k, L, \perp, r \rangle} \quad [\text{DRP}]$$

$$\xrightarrow{1} \langle p, pc+1, s', dev, cap, k+1, L, \perp, r \rangle$$

Stack underflow (general). For any stack-consuming instruction when the stack has insufficient elements:

$$\frac{p[pc] \in \{\text{IOW, EQ, SUB, DRP, JZ, DUP}\} \quad k < L \quad \text{stack underflow}}{\langle p, pc, s, dev, cap, k, L, \perp, r \rangle} \quad [\text{POPF}]$$

$$\xrightarrow{1} \langle p, pc+1, s, dev, cap, k+1, L, \text{POP}, r \rangle$$

Invalid opcode.

$$\frac{p[pc] \notin \text{Opcodes} \quad k < L}{\langle p, pc, s, dev, cap, k, L, \perp, r \rangle} \quad [\text{OPCF}]$$

$$\xrightarrow{1} \langle p, pc+1, s, dev, cap, k+1, L, \text{OPC}, r \rangle$$

READBACK d . Capability-gated device read (same precondition as IOR-OK):

$$\frac{p[pc] = \text{READBACK } d \quad k < L \quad d \in cap}{\langle p, pc, s, dev, cap, k, L, \perp, r \rangle} \quad [\text{READBACK-OK}]$$

$$\xrightarrow{1} \langle p, pc+1, s \cdot dev(d), dev, cap, k+1, L, \perp, r \rangle$$

$$\frac{p[pc] = \text{READBACK } d \quad k < L \quad d \notin cap}{\langle p, pc, s, dev, cap, k, L, \perp, r \rangle} \quad [\text{READBACK-CAPF}]$$

$$\xrightarrow{1} \langle p, pc+1, s, dev, cap, k+1, L, \text{CAP}, r \rangle$$

RETRY n o. The retry counter r tracks remaining iterations. When $r > 0$, decrement r and jump to offset o ; when $r = 0$, fall through. The compiler initializes $r \leftarrow n$ before entering the retry block (see Appendix B.1).

$$\frac{p[pc] = \text{RETRY } n \ o \quad k < L \quad r > 0}{\langle p, pc, s, dev, cap, k, L, \perp, r \rangle} \quad [\text{RETRY-LOOP}]$$

$$\xrightarrow{1} \langle p, pc+o, s, dev, cap, k+1, L, \perp, r-1 \rangle$$

$$\frac{p[pc] = \text{RETRY } n \ o \quad k < L \quad r = 0}{\langle p, pc, s, dev, cap, k, L, \perp, 0 \rangle} \quad [\text{RETRY-EXHAUST}]$$

$$\xrightarrow{1} \langle p, pc+1, s, dev, cap, k+1, L, \perp, 0 \rangle$$

Remark 2 (Extended formal model). The 17 rules above (LIT through OpCF, plus READBACK and RETRY) cover the 14-instruction set. The two additional rules (READBACK-OK/CAPF, RETRY-LOOP/EXHAUST) extend the model to cover the closed-loop subset. READBACK inherits the same capability safety guarantee as IOR-OK/IO-CAPF. RETRY preserves bounded termination because: (i) each RETRY-LOOP transition increments k by 1, so the global step counter remains monotonic; (ii) the retry counter r is strictly decreasing ($r' = r - 1$) per loop iteration, and r is finite (initialized by the compiler to the constant n from the RETRY instruction's immediate operand); (iii) execution terminates when either $k \geq L$ (STEPGUARD) or $r = 0$ (RETRY-EXHAUST). Determinism holds because READBACK returns a deterministic function of $dev(d)$ and RETRY depends only on the current value of r . Propositions 1–3 therefore extend to the 14-instruction set (12 sequential + READBACK + RETRY) without modification to the proof structure.

C.2. Supplementary Experiment Results

Error type classification. Across all task sets, first-attempt failures fall into two categories. The dominant type is LIR_PARSE_ERROR (occasional LLM syntax deviations): 11 out of 113 tasks, 1 out of 15 control-flow tasks, and 5 out of 96 closed-loop tasks. The second type is TASK_SIG_MISMATCH (the program compiles and executes but the resulting device state does not match expectations): 2 out of 113 tasks on the deterministic set. On the closed-loop set under a basic

Table 13

Extended task-set generation results (temperature=0.0): control-flow set (tasks_v4, $n=15$) and extended closed-loop set (tasks_v3_expanded, $n=96$).

Metric	Control-flow ($n=15$)		Closed-loop ($n=96$)	
	Value	95% Wilson CI	Value	95% Wilson CI
IR validity rate	0.933	[0.698, 0.997]	0.948	[0.886, 0.979]
Compile pass rate	0.933	[0.698, 0.997]	0.948	[0.886, 0.979]
Task success rate	0.867	[0.621, 0.963]	0.948	[0.886, 0.979]
Failed tasks	2 / 15	—	5 / 96	—
Primary failure	—	—	LIR_PARSE_ERROR (5)	

Table 14

Multi-model comparison: generation results on 113 deterministic tasks and 600 random tasks (temperature=0.0). Random-600 TSR for deepseek-v4-pro is newly measured in this work; others are llama3.1:8b only.

Model	Type	IR	Compile	TSR-113	95% CI	TSR-600	Errors
llama3.1:8b [14]	Local 8B	1.000	1.000	1.000	[0.967, 1.000]	0.970	—
qwen3:8b	Local 8B	0.956	0.956	0.903	[0.835, 0.946]	—	SIG(6)+PARSE(5)
deepseek-r1:8b [20]	Local 8B	1.000	1.000	1.000	[0.967, 1.000]	—	—
deepseek-v4-pro [13]	Commercial API	0.991	0.991	0.991	[0.950, 0.999]	0.987	PARSE(1)

Table 15

Negative baseline comparison: JSON + Schema + external compression vs. LIR + compiler (113 deterministic tasks, llama3.1:8b, temp=0.0).

Metric	JSON+zlib	LIR bytecode
Validity rate	1.000 [0.967, 1.000]	1.000 [0.967, 1.000]
Task success rate	0.965 [0.913, 0.986]	1.000 [0.967, 1.000]
Avg. raw output (bytes)	156.4	—
Avg. zlib level 9 (bytes)	106.7	—
Avg. CBOR (bytes)	105.2	—
Avg. MessagePack (bytes)	97.42	—
Avg. compact JSON (bytes)	127.18	—
Avg. bytecode (bytes)	—	10.9
vs. zlib-JSON ratio	—	9.8x
vs. CBOR ratio	—	9.6x
vs. MessagePack ratio	—	8.9x
vs. compact JSON ratio	—	11.7x
Primary failure	TASK_SIG(4)	—
Direct Hex	0.000	—

The raw/zlib/CBOR rows are measured on a hand-crafted minimal-schema JSON (raw 156.4 B); the MessagePack and compact-JSON rows are measured on the raw-LLM-output JSON (raw 285.9 B), so the two baselines differ in schema. Ratios are computed against the bytecode within each respective baseline.

prompt without few-shot examples, an additional 41 tasks exhibited TASK_SIG_MISMATCH, of which 34 were a specific sub-class where the LLM generated repeat instead of retry. Under the deployed few-shot prompt with the bounded repair loop, all of these first-attempt and basic-prompt failures are resolved (113-task TSR = 1.000).

C.3. Scalability Analysis

We evaluated LIR pipeline scalability along three axes: task length, device count, and step-limit sensitivity. All measurements were conducted on the host-side simulator, which reproduces the LVM execution model faithfully including capability checks and step counting.

Task length. We generated synthetic LIR programs with 1–50 operations using alternating set/read/wait patterns over 2 devices. Fig. 9 reports bytecode size and execution steps as a function of operation count. Both metrics exhibit strictly linear growth: bytecode size at ≈ 3.33 B per operation (regression slope, $R^2 > 0.999$), and execution steps at ≈ 1.67 steps

Table 16

Random task generation, 11ama3.1:8b per-category breakdown ($n=600$, temperature,=,0.0). Aggregate metrics are reported in Section 7.4.4.

Category	Pass/Total	Success Rate
Single write	50/50	1.000
Single read	50/50	1.000
Write-wait-read	50/50	1.000
Pulse	50/50	1.000
Pulse with read	50/50	1.000
Multi-device write	50/50	1.000
Write-read-multi	50/50	1.000
Write-wait-halt	46/50	0.920
Write-wait-read (multi)	45/50	0.900
Multiple reads	48/50	0.960
Write-wait-multi-read	45/50	0.900
Long sequence	48/50	0.960

Table 17

Real-world IoT task evaluation, per-category retry/readback breakdown ($n=29$, temperature,=,0.0). Aggregate metrics are in Section 7.4.5, Table 10. Rows with $n \leq 2$ are reported for completeness; the aggregate subsets carry the statistical weight.

Category	LIR v1	LIR v4	JSON
Retry-verify (4)	0/4	2/4	3/4
Retry-watering (1)	0/1	0/1	1/1
Retry with readback (1)	0/1	1/1	1/1
Safety shutoff (1)	0/1	0/1	1/1
Zone cycle (1)	0/1	1/1	0/1
Pulse with verify (1)	0/1	1/1	0/1
Multi-device sequence (1)	0/1	1/1	1/1

per operation. Compilation time remains below $300 \mu s$ even at 50 operations. This confirms that the deterministic compiler C operates in $O(n)$ time and that the bytecode does not suffer from super-linear bloat as task complexity increases.

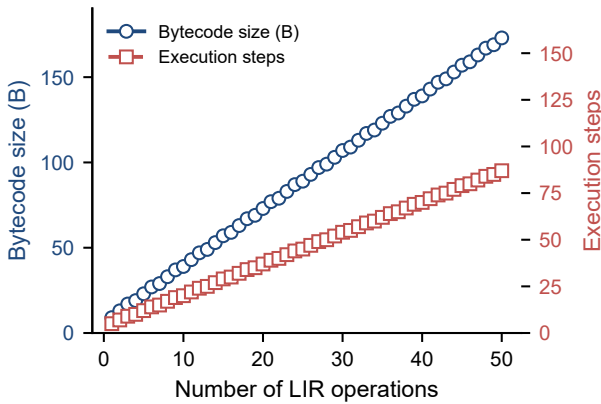


Figure 9: Bytecode size and execution steps vs. operation count.

Device count. We varied the number of distinct devices from 2 to 6 while holding the operation count fixed at 10. Fig. 10 shows that bytecode size and execution steps remain essentially flat (35–39 B, 18–20 steps). Each additional device adds only a single GTWAY instruction (~ 2 B) and one I/O operation, so the marginal payload cost per device is

negligible. This confirms that LIR’s per-device overhead is bounded and that the framework scales to multi-device scenarios without payload explosion.

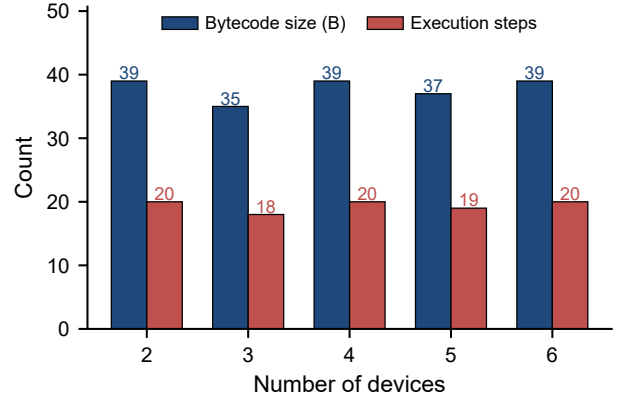


Figure 10: Bytecode size and execution steps vs. device count.

Step-limit sensitivity. Using four tasks of varying complexity (9–54 base execution steps), we swept the step-limit L from 10 to 1000 and recorded whether each task completed under the limit. Fig. 11 shows that all tasks complete when L exceeds their base step count, and fail safely when L is below the required minimum. The default $L = 1000$ used throughout this paper provides a $10\times$ – $100\times$ safety margin over the measured execution steps (9–54), ensuring that even adversarial programs with worst-case loop behavior terminate within the bound (Proposition 3) while imposing negligible runtime overhead on the MCU.

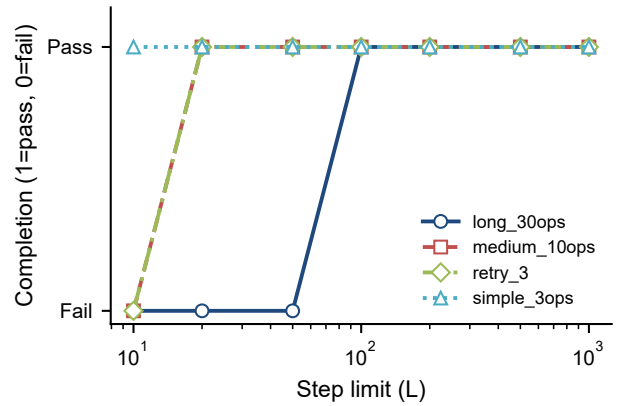


Figure 11: Task completion vs. step-limit L across tasks of varying base step count.

C.4. Full LVM Instruction Set

Table 18 gives the complete instruction set of the host-side LVM reference implementation. The 14-instruction bounded-execution profile used in this paper (Table 5) is drawn from this set; the remaining instructions are implemented and unit-tested but excluded from the on-device safety analysis (Section 5.4).

Table 18

LVM instruction set in the host-side reference implementation, grouped by category.

Category	Instructions
Control flow	FN, CALL, RET, IF, WHILE, FOR, JMP, JZ, JNZ
Stack & values	LIT, VAR, LET, SET, DUP, DRP, SWP, ROT
Comparison	LT, GT, LE, GE, EQ, NEQ
Arithmetic & bitwise	ADD, SUB, MUL, DIV, MOD, NEG, NOT, AND, OR, XOR, SHL, SHR
Arrays & memory	NEWARR, GET, PUT, IDX, STO, LEN, ALLOC, FREE
I/O & system	GTWAY, IOW, IOR, WAIT, HALT, TRACE, GC, BP, STEP

C.5. Supplementary Analysis

C.5.1. Prompt caching assumptions and limitations

The cache dependency analysis is presented in Section 7.2.3. The full three-way token detail (LIR, JSON, Python), including the cached incremental cost (output-token column), is given in Table 12.

C.6. Covered and Uncovered Task Type Boundaries

The verified task types span single-device read/write, wait-terminate, wait-read, multi-device read, pulse sequences (113-task set), readback verification and retry loops (96 closed-loop tasks), conditional branching and loop control (15-task set, TSR = 0.867), and real-world sequential control and retry/readback patterns (29-task set, TSR = 0.793). Unverified types include nested closed-loop constructs (retry{readback{retry{...}}}), error-handling branches (on fault(code){...}), and strong real-time closed-loop control (periodic PID). These unverified types require either extended grammar support or real-time guarantees that lie outside the current scope.

References

- [1] Al-Fuqaha, A., Guizani, M., Mohammadi, M., et al., 2015. Internet of things: A survey on enabling technologies, protocols, and applications. *IEEE Commun. Surv. Tutor.* 17, 2347–2376. doi:10.1109/COMST.2015.2444095.
- [2] atomvm, 2024. Atomvm: A compact virtual machine for erlang on microcontrollers. <https://github.com/atomvm/AtomVM>.
- [3] gVisor Authors, 2024. gvisor: Application kernel for containers. <https://gvisor.dev>.
- [4] Baccelli, E., Hahm, O., Gunes, M., et al., 2013. RIOT OS: Towards an os for the internet of things, in: *Proc. IEEE INFOCOM*.
- [5] Bormann, C., Ersue, M., Keranen, A., 2014. Terminology for constrained-node networks. doi:10.17487/RFC7228. rFC 7228.
- [6] Bormann, C., Hoffman, P., 2020. Concise binary object representation (cbor). doi:10.17487/RFC8949. rFC 8949.
- [7] Brohan, A., Brown, N., Carbajal, J., et al., 2023. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*.
- [8] Brouwers, N., Langendoen, K., Corke, P., 2009. Darjeeling, a feature-rich VM for the resource poor, in: *Proc. ACM SenSys*. doi:10.1145/1644038.1644056.
- [9] bytecodealliance, 2024. Webassembly micro runtime (wamr). <https://github.com/bytecodealliance/wasm-micro-runtime>.
- [10] Carlini, N., Tramèr, F., Wallace, E., Jagielski, M., Herbert-Voss, A., Lee, K., Roberts, A., Brown, T., Song, D., Erlingsson, U., Oprea, A., Raffel, C., 2021. Extracting training data from large language models, in: 30th USENIX Security Symposium (USENIX Security 21), pp. 2633–2650.
- [11] Chase, H., 2024. Langchain: Building applications with llms. <https://github.com/langchain-ai/langchain>.
- [12] Chen, M., Tworek, J., Jun, H., et al., 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- [13] DeepSeek-AI, 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*.
- [14] Dubey, A., et al., 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- [15] Esposito, P., Ferrandi, M.L., et al., 2024. Llm-assisted embedded software development: A systematic review. *J. Syst. Archit.* 152, 103155.
- [16] eBPF Foundation, 2024. ebpf: Extended berkeley packet filter. <https://ebpf.io>.
- [17] Geden, M., Rasmussen, K., 2023. Hardware-assisted remote attestation design for critical embedded systems. *IET Inf. Secur.* 17. doi:10.1049/ise2.12113.
- [18] Geng, S., Joshi, R., Bhatt, S., et al., 2024. Grammar-constrained decoding for structured llm outputs, in: *Proc. ACL*.
- [19] George, D.P., 2014. Micropython: Python for microcontrollers. <https://micropython.org>.
- [20] Guo, D., Zhu, Q., Yang, D., et al., 2024. Deepseek-coder: When the large language model meets programming—the rise of code intelligence. *arXiv preprint arXiv:2401.14196*.
- [21] jakogut, 2024. Tinyvm: A small, fast, lightweight virtual machine written in pure ansi c. <https://github.com/jakogut/tinyvm>.
- [22] Kim, M., Pertsch, K., Karamcheti, S., et al., 2024. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*.
- [23] Klein, G., Elphinstone, K., Heiser, G., et al., 2009. sel4: Formal verification of an os kernel, in: *Proc. ACM SOSP*. doi:10.1145/1629575.1629596.
- [24] Leroy, X., 2009. Formal verification of a realistic compiler. *Commun. ACM* 52, 107–115. doi:10.1145/1538788.1538814.
- [25] Li, Y., Choi, D., Chung, J., et al., 2022. Competition-level code generation with alphacode. *Science* 378, 1092–1097.
- [26] Liang, J., Huang, W., Xia, F., et al., 2023. Code as policies: Language model programs for embodied control, in: *Proc. IEEE ICRA*. doi:10.1109/ICRA48891.2023.10160591.
- [27] Linux Kernel Documentation, 2024. seccomp bpf. https://www.kernel.org/doc/html/latest/userspace-api/seccomp_filter.html.
- [28] Liu, M., Pinckney, N., Khailany, B., et al., 2023. Verilogval: Evaluating large language models for verilog code generation, in: *Proc. ICCAD*.
- [29] Lua.org, 2024. Lua programming language. <https://www.lua.org/>.
- [30] MessagePack, 2024. An efficient binary serialization format. <https://msgpack.org>.
- [31] OASIS, 2014. Mqtt version 3.1.1. OASIS Standard. URL: <http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/>.
- [32] Patil, S., Zhang, T., Wang, X., Gonzalez, J.E., 2023. Gorilla: Large language model connected with massive apis, in: *arXiv preprint arXiv:2305.15334*.
- [33] Project, Z., 2024. Zephyr: A scalable real-time operating system. <https://zephyrproject.org>.
- [34] Raffel, C., Shazeer, N., Roberts, A., et al., 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.* 21, 1–67.
- [35] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., Scialom, T., 2023. Toolformer: Language models can teach themselves to use tools, in: *Proc. NeurIPS*.
- [36] Schoeberl, M., et al., 2014. Safety-critical java: The mission to improve safety in real-time java, in: *JTRES*.
- [37] Serpanos, D.N., Voyiatzis, A.G., 2013. Security challenges in embedded systems. *ACM Trans. Embed. Comput. Syst.* 12, 1–10. doi:10.1145/2435227.2435262.
- [38] Services, A.W., 2024. Freertos: Real-time operating system for microcontrollers. <https://www.freertos.org>.
- [39] Shelby, Z., Hartke, K., Bormann, C., 2014. The constrained application protocol (coap). doi:10.17487/RFC7252. rFC 7252.

- [40] Solouki, M.A., Angizi, S., Violante, M., 2024. Dependability in embedded systems: A survey of fault tolerance methods and software-based mitigation techniques. *IEEE Access* 12. doi:10.1109/access.2024.3509633.
- [41] Watson, R.N.M., Woodruff, J., Neumann, P.G., Moore, S.W., Anderson, J., Chisnall, D., Dave, N., Davis, B., Gudka, K., Laurie, B., Murdoch, S.J., Norton, R., Roe, M., Son, S.D., Vadera, M., 2015. Cheri: A hybrid capability-system architecture for scalable software compartmentalization, in: *Proc. IEEE S&P*. doi:10.1109/SP.2015.9.
- [42] Willard, R., Lefebvre, L., 2023. Outlines: Structured text generation. <https://github.com/dottxt-ai/outlines>.
- [43] Yee, B., et al., 2009. Native client: A sandbox for portable, untrusted x86 native code, in: *IEEE S&P*.